

**Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Санкт-Петербургский государственный университет
промышленных технологий и дизайна»**

Институт информационных технологий и автоматизации
Направление подготовки 09.03.02 – Информационные системы и технологии
(Информационные системы и сетевые технологии)
Выпускающая кафедра интеллектуальных систем и защиты информации

Допустить к защите
Заведующий кафедрой проф. Макаров А.Г.
" " _____ 2026 г.

**ВЫПУСКНАЯ
КВАЛИФИКАЦИОННАЯ РАБОТА**
(бакалаврская работа)

на тему: «Система метапоиска товаров на торговых площадках
с функцией интеллектуального анализа цен»

Исполнитель - студент учебной группы 4-МД-16

Санько Егор Александрович

Руководитель выпускной квалификационной работы

к.т.н. Вагнер Виктория Игоревна

Нормоконтролер: к.т.н. Чалова Екатерина Игоревна

**Санкт-Петербург
2026**

РЕФЕРАТ

Выпускная квалификационная работа содержит 93 страницы, 18 рисунков, 23 таблицы, 25 использованных источников, 2 приложения.

КЛЮЧЕВЫЕ СЛОВА: МЕТАПОИСК, ПАРСИНГ, МАРКЕТПЛЕЙС, ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ, СРАВНЕНИЕ ЦЕН, СКРАПИНГ, ВЕБ-ПРИЛОЖЕНИЕ, *FASTAPI*, *NEXT.JS*, *GEMINI*, *OPENROUTER*, *SSE*, ПОТОКОВАЯ ПЕРЕДАЧА ДАННЫХ, ИСТОРИЯ ЦЕН, АГРЕГАТОР ЦЕН, ЭЛЕКТРОННАЯ КОММЕРЦИЯ, *DOCKER*, *POSTGRESQL*, АНТИБОТ-ЗАЩИТА.

Объект исследования — процессы поиска и сравнения цен на товары на маркетплейсах России и Беларуси; предмет исследования — методы автоматизированного метапоиска и интеллектуального анализа цен. Цель работы — проектирование и разработка веб-платформы метапоиска с интеграцией *AI*-ассистента на базе модели *Gemini 2.5 Flash* и потоковой передачей результатов через *Server-Sent Events*.

В первой главе проанализированы рынок электронной коммерции России и Беларуси и существующие решения, выявлены функциональные пробелы. Во второй главе спроектирована архитектура системы: модель базы данных, *REST API*, *SSE*-стриминг и контейнеризация в *Docker*. В третьей главе реализованы парсеры маркетплейсов, *AI*-коррекция поисковых запросов и серверная часть на *FastAPI*. В четвёртой главе проведены функциональное и нагрузочное тестирование и оптимизация производительности.

В результате разработана и развёрнута в промышленную эксплуатацию система *Smart Price* (*smrt-price.ru*), обеспечивающая мгновенный поиск, сравнение цен на маркетплейсах и *AI*-рекомендации по выбору товаров.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ	12
1.1 Рынок электронной коммерции и маркетплейсов	12
1.2 Анализ существующих решений	18
1.3 Технологический обзор	21
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МЕТАПОИСКА	29
2.1 Анализ требований	29
2.2 Архитектурное проектирование	31
2.3 Проектирование базы данных	34
2.4 Проектирование <i>API</i>	36
2.5 Проектирование пользовательского интерфейса	39
2.6 Проектирование развёртывания	41
2.7 Проектирование обработки ошибок и отказоустойчивости	42
2.8 Проектирование кэширования	43
3 РЕАЛИЗАЦИЯ СИСТЕМЫ МЕТАПОИСКА	45
3.1 Реализация парсеров маркетплейсов	45
3.2 Реализация <i>AI</i> -коррекции запросов	47
3.3 Реализация гибридной системы фильтрации	48
3.4 Реализация <i>AI</i> -ассистента	51
3.5 Реализация <i>SSE</i> -стриминга	53
3.6 Реализация фронтенда	55
3.7 Интеграция с <i>Gemini</i> через <i>OpenRouter</i> [14]	57

3.8 Реализация системы аутентификации.....	58
3.9 Реализация прокси-сервера изображений	60
3.10 Реализация модуля истории цен.....	60
3.11 Реализация AI-анализа цен	61
3.12 Развёртывание и <i>DevOps</i>	64
3.13 Реализация функции поиска более выгодных предложений («Найти дешевле»)	65
4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ.....	69
4.1 Методология тестирования	69
4.2 Результаты функционального тестирования.....	70
4.3 Результаты тестирования AI-ассистента.....	72
4.4 Результаты тестирования производительности	73
4.5 Сквозное тестирование интерфейса	75
4.6 Тестирование адаптивной вёрстки	79
4.7 Сравнение с существующими решениями	80
4.8 Экономический анализ эксплуатации.....	81
ЗАКЛЮЧЕНИЕ	85
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	88
ПРИЛОЖЕНИЯ.....	90

ВВЕДЕНИЕ

Стремительное развитие электронной коммерции в последние годы кардинально изменило модель потребительского поведения. По данным Ассоциации компаний интернет-торговли (АКИТ), объём российского рынка электронной коммерции в 2023 году составил 6,4 триллиона рублей, продемонстрировав рост на 28 процентов по сравнению с предыдущим годом. В Республике Беларусь наблюдается аналогичная тенденция: рынок онлайн-торговли активно расширяется, а количество активных покупателей стабильно увеличивается год от года. По прогнозам аналитических агентств, к 2027 году совокупный объём рынка электронной коммерции в России превысит 15 триллионов рублей, а доля онлайн-продаж в общем розничном обороте достигнет 25 процентов. Данные показатели свидетельствуют о фундаментальной трансформации потребительских привычек, при которой традиционная модель посещения физических магазинов всё в большей степени уступает место электронным каналам приобретения товаров [2].

Параллельно с ростом рынка происходит усложнение его структуры. На территории двух стран функционируют десятки крупных торговых площадок: *Wildberries*, *Ozon* и Яндекс Маркет в России, *Onliner.by* в Беларуси, а также множество специализированных магазинов. Каждая платформа обладает уникальным каталогом, собственной системой ценообразования, различными механизмами скидок и промоакций. В результате потребитель оказывается в ситуации информационной асимметрии, описанной ещё в классической работе Дж. Акерлофа: располагая неполной информацией о реальном уровне цен на рынке, покупатель вынужден принимать решения в условиях неопределённости [1]. Масштаб данной проблемы возрастает пропорционально количеству торговых площадок и номенклатуре представленных товаров. Особенно остро проблема проявляется в категориях дорогостоящей электроники и

бытовой техники, где абсолютная разница в ценах может составлять десятки тысяч рублей.

Ситуацию усугубляет практика динамического ценообразования, широко применяемая современными маркетплейсами. Цена на один и тот же товар может различаться на десятки процентов в зависимости от площадки, времени суток, истории просмотров пользователя и иных факторов. Алгоритмы ценообразования учитывают текущий спрос, уровень конкуренции, сезонность продаж, а также поведенческие паттерны конкретного пользователя. Некоторые маркетплейсы применяют практику персонализированного ценообразования, при которой разные покупатели видят различные цены на один и тот же товар в зависимости от их истории покупок, региона и используемого устройства. Ручное сравнение цен требует значительных временных затрат: необходимо открыть несколько сайтов, ввести одинаковый запрос, учесть различия в наименованиях, сопоставить характеристики и условия доставки. По экспертным оценкам, среднестатистический покупатель тратит от 30 до 90 минут на сравнение цен при совершении крупной покупки, причём далеко не всегда находит оптимальное предложение.

Дополнительным фактором, повышающим актуальность исследования, является кросс-региональный характер потребления в едином торговом пространстве России и Беларуси. Жители Беларуси активно приобретают товары на российских маркетплейсах благодаря единому таможенному пространству Евразийского экономического союза и относительно невысоким логистическим затратам. *Wildberries* и *Ozon* открыли сети пунктов выдачи заказов в крупных белорусских городах, что существенно упростило процесс трансграничных покупок. Однако существующие агрегаторы цен ориентированы преимущественно на один регион и одну валюту, что не позволяет в полной мере реализовать потенциал трансграничной торговли. Покупатель из Минска вынужден самостоятельно сопоставлять предложения белорусских магазинов в белорусских рублях с

предложениями российских маркетплейсов в российских рублях, выполняя конвертацию валют и сравнивая условия доставки в ручном режиме.

Развитие технологий искусственного интеллекта и больших языковых моделей открывает принципиально новые возможности для создания интеллектуальных систем поддержки принятия решений в сфере электронной коммерции. Современные модели, такие как *Gemini* от компании *Google* и *Claude* от компании *Anthropic*, поддерживают механизм инструментального взаимодействия (*tool use*), позволяющий *AI*-агенту выполнять действия во внешних системах — осуществлять поиск, анализировать данные и формулировать обоснованные рекомендации на естественном языке. Данный механизм превращает языковую модель из пассивного генератора текста в активного агента, способного самостоятельно принимать решения о необходимости и параметрах поиска. В контексте метапоиска товаров это означает, что пользователь может сформулировать запрос на естественном языке, а *AI*-агент самостоятельно определит оптимальную стратегию поиска, выполнит необходимые запросы и представит результаты с аналитическим комментарием [10].

Анализ существующих решений в области сравнения цен — *Google Shopping*, *Ideal*o, *e-katalog.ru*, *PriceRunner* и других — показывает, что ни одно из них не обеспечивает одновременно кроссрегионального покрытия Беларуси и России, потокового отображения результатов в реальном времени через *Server-Sent Events* и интеллектуального *AI*-ассистента для анализа цен с механизмом *tool use*. Зарубежные агрегаторы не покрывают рынки стран СНГ, российские решения не включают белорусские площадки, а белорусские агрегаторы ограничены локальным рынком. Данный пробел определяет актуальность разработки нового решения, способного объединить преимущества всех существующих подходов и дополнить их возможностями искусственного интеллекта.

Актуальность работы обусловлена также техническими вызовами, связанными с извлечением данных из маркетплейсов. Современные торговые

площадки применяют многоуровневую антибот-защиту, включающую *CAPTCHA*, *TLS*-фингерпринтинг, анализ поведенческих паттернов и ограничение частоты запросов. Преодоление данных барьеров без использования ресурсоёмких *headless*-браузеров представляет собой научно-техническую задачу, решение которой имеет как практическое, так и теоретическое значение. В рамках данной работы предложены и реализованы оригинальные методы извлечения данных, в частности метод *regex*-извлечения из встроеного *JSON*-содержимого *HTML*-страниц Яндекс Маркет, обеспечивающий десятикратное ускорение по сравнению с традиционным подходом на основе *headless*-браузеров.

Целью данной работы является проектирование и разработка веб-платформы метапоиска товаров на маркетплейсах Беларуси и России с *AI*-анализом цен, обеспечивающей пользователям единый интерфейс для поиска, сравнения и мониторинга цен на товары с интеллектуальными рекомендациями на базе большой языковой модели *Gemini 2.5 Flash*. Платформа должна объединять данные из пяти маркетплейсов — *Onliner.by*, Яндекс Маркет, *Wildberries*, Ретард и *World Devices* — и предоставлять результаты в потоковом режиме реального времени через технологию *Server-Sent Events*.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) Провести анализ предметной области электронной коммерции и существующих решений для сравнения цен на маркетплейсах, выявить функциональные пробелы существующих сервисов и определить конкурентные преимущества разрабатываемого решения.

- 2) Исследовать методы и технологии веб-скрапинга с учётом современных механизмов антибот-защиты маркетплейсов, определить оптимальные стратегии извлечения данных для каждой целевой площадки.

3) Спроектировать архитектуру распределённой системы метапоиска с поддержкой потоковой передачи данных на основе *Server-Sent Events* и контейнерного развёртывания.

4) Разработать специализированные модули парсинга данных с маркетплейсов *Onliner.by*, Яндекс Маркет, *Wildberries*, Ретард и *World Devices* с учётом их технических особенностей и механизмов защиты.

5) Реализовать серверную часть приложения на базе фреймворка *FastAPI* с поддержкой *SSE*-стриминга, асинхронного взаимодействия с *PostgreSQL* через *SQLAlchemy 2.0* [17] и *asyncpg*.

6) Разработать клиентскую часть приложения на базе фреймворка *Next.js 14* с *App Router*, обеспечив адаптивный пользовательский интерфейс с тёмной темой.

7) Интегрировать *AI*-ассистента на основе *Gemini 2.5 Flash* (через *OpenRouter*) с инструментальным взаимодействием (*tool use*) для интеллектуального анализа цен и формулирования рекомендаций.

8) Реализовать систему отслеживания и визуализации истории цен с агрегированной статистикой и графическим представлением данных.

9) Провести комплексное тестирование и оптимизацию производительности разработанной системы, включая модульное, интеграционное и нагрузочное тестирование.

10) Развернуть систему на виртуальном выделенном сервере с использованием *Docker Compose* и обеспечить её функционирование в промышленных условиях с надлежащим уровнем безопасности.

Объектом исследования является процесс поиска и сравнения цен на товары в сети Интернет в условиях множественности торговых площадок и динамического ценообразования. Предметом исследования выступают методы и технологии автоматизированного агрегирования ценовой информации с торговых площадок с применением технологий искусственного интеллекта для анализа и интерпретации полученных данных.

Научная новизна работы заключается в следующем. Во-первых, предложен метод извлечения структурированных данных из Яндекс Маркет с помощью регулярных выражений, применяемых к встроенному *JSON*-содержимому *HTML*-страниц, обеспечивающий десятикратное ускорение парсинга по сравнению с традиционным подходом на основе *headless*-браузеров (1–2 секунды против 6–10 секунд). Во-вторых, впервые реализована интеграция *AI*-ассистента на базе *Gemini 2.5 Flash* (через *OpenRouter*) с механизмом *tool use* в контексте кроссрегионального метапоиска товаров, позволяющая пользователю взаимодействовать с системой на естественном языке. В-третьих, предложена гибридная стратегия фильтрации результатов (*Stage 1 regex + Stage 2 AI*), обеспечивающая семантическое понимание контекста запроса при минимальных затратах на *AI*-обработку.

Практическая значимость работы определяется тем, что система развёрнута и функционирует в промышленных условиях на виртуальном выделенном сервере. Эмпирические исследования показали, что разница в ценах на идентичные товары между площадками составляет от 5 до 40 процентов, что делает использование системы экономически оправданным. Для покупки стоимостью 50 000 рублей потенциальная экономия составляет от 6 000 до 9 000 рублей, что многократно превышает эксплуатационные расходы на содержание системы.

Методы исследования включают системный анализ для декомпозиции предметной области и определения требований, объектно-ориентированное проектирование с применением паттернов проектирования (Стратегия, Фасад, Наблюдатель), методы веб-скрапинга и обратного инжиниринга недокументированных *API*, технологии потоковой передачи данных на основе протокола *SSE*, методы обработки естественного языка с использованием больших языковых моделей, экспериментальное тестирование с использованием *pytest* и профилирование производительности.

Работа состоит из введения, четырёх глав, заключения, списка использованных источников и двух приложений. В первой главе проведён анализ предметной области электронной коммерции, исследованы существующие решения для сравнения цен, выполнен обзор технологий. Во второй главе представлено проектирование архитектуры системы, базы данных, *API* и *AI*-подсистемы. В третьей главе описана реализация компонентов системы, включая парсеры, сервер, *AI*-ассистента, клиентскую часть и инфраструктуру. В четвёртой главе приведены результаты тестирования, анализ производительности, безопасности и экономическая оценка проекта.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ

1.1 Рынок электронной коммерции и маркетплейсов

Современное состояние рынка *e-commerce* в России. Электронная коммерция является одним из наиболее динамично развивающихся секторов мировой экономики. Согласно данным Ассоциации компаний интернет-торговли (АКИТ), совокупный оборот рынка электронной коммерции в России в 2023 году достиг 6,4 триллиона рублей, продемонстрировав рост на 28 процентов по сравнению с предыдущим годом. Количество выполненных заказов составило более 5,03 миллиарда единиц, а число активных покупателей превысило 70 миллионов человек. Средний чек онлайн-покупки составил порядка 1230 рублей, однако в отдельных категориях, таких как электроника и бытовая техника, средний чек достигает 15–20 тысяч рублей [2].

Ведущие позиции на рынке занимают маркетплейсы — многосторонние торговые платформы, выступающие посредниками между продавцами и покупателями. По состоянию на 2024 год, крупнейшим маркетплейсом России является *Wildberries* с долей рынка порядка 47 процентов и годовым *GMV* (*Gross Merchandise Value*) свыше 2,5 триллиона рублей. Второе место занимает *Ozon* с долей около 34 процентов и *GMV* порядка 1,75 триллиона рублей. Яндекс Маркет, интегрированный в экосистему Яндекса, занимает порядка 8 процентов рынка, однако демонстрирует наиболее агрессивный темп роста — более 50 процентов в годовом исчислении [3].

Структура рынка характеризуется высокой степенью концентрации: на долю трёх крупнейших маркетплейсов приходится более 85 процентов совокупного оборота. Данная олигополистическая структура создаёт ситуацию, при которой потребитель фактически обязан проверять цены на нескольких площадках для принятия оптимального решения о покупке.

Каждый маркетплейс стремится привлечь покупателей собственными программами лояльности, системами кэшбэка и промоакциями, что приводит к существенным различиям в итоговой стоимости одного и того же товара на разных площадках.

Категорийная структура российского *e-commerce* существенно изменилась за последние годы. Если в 2019 году доминирующей категорией была электроника и бытовая техника (28 процентов), то к 2023 году лидирующие позиции заняли товары повседневного спроса (*FMCG*) с долей 35 процентов. Тем не менее, именно в категориях электроники, смартфонов, ноутбуков и бытовой техники наблюдается наибольший ценовой разброс между площадками, что делает данные категории приоритетными для системы метапоиска. Разница в цене на идентичный товар может составлять от 5 до 40 процентов, что в абсолютном выражении означает экономию от нескольких тысяч до десятков тысяч рублей.

Рынок электронной коммерции в Беларуси. В Республике Беларусь безусловным лидером рынка электронной коммерции является портал *Onliner.by*, основанный в 2001 году и представляющий собой уникальный для постсоветского пространства агрегатор товарных предложений. Ежемесячная аудитория портала превышает 3 миллиона уникальных посетителей, что составляет более трети населения страны. В отличие от российских маркетплейсов, *Onliner* не является непосредственной торговой площадкой: он агрегирует предложения от сотен интернет-магазинов, предоставляя покупателям возможность сравнить цены и перейти на сайт продавца для оформления заказа. Данная модель обеспечивает высокую степень ценовой прозрачности на белорусском рынке [4].

Белорусский рынок электронной коммерции характеризуется активным проникновением российских маркетплейсов. *Wildberries* и *Ozon* открыли пункты выдачи заказов в крупных городах Беларуси, а белорусские покупатели всё чаще совершают покупки на российских площадках, используя преимущества единого таможенного пространства ЕАЭС. Объём

трансграничной торговли демонстрирует ежегодный рост порядка 15–20 процентов. Вместе с тем белорусские покупатели сталкиваются с необходимостью сопоставления цен в разных валютах — белорусских и российских рублях, что существенно усложняет процесс принятия решения. Курсовые колебания могут существенно влиять на выгодность той или иной покупки, особенно при приобретении дорогостоящих товаров.

Особенностью белорусского рынка является также наличие развитой системы каталогизации товаров на *Onliner.by*. Каждый товар снабжён подробными техническими характеристиками, отзывами покупателей, историей цен и списком предложений от различных продавцов. Это создаёт уникальную информационную среду, данные из которой представляют значительную ценность для системы метапоиска. Техническая реализация *Onliner* предполагает наличие открытого *JSON API* для каталога товаров, что существенно упрощает интеграцию и извлечение данных по сравнению с российскими маркетплейсами, применяющими агрессивную антибот-защиту.

Тенденции развития рынка *e-commerce*. К ключевым тенденциям развития рынка электронной коммерции в регионе относятся следующие. Во-первых, доминирование маркетплейс-модели: по прогнозам аналитиков, к 2026 году доля маркетплейсов превысит 80 процентов совокупного оборота электронной коммерции, вытеснив монобрендовые интернет-магазины. Во-вторых, стремительный рост мобильной коммерции: до 70 процентов заказов уже совершаются с мобильных устройств, что предъявляет повышенные требования к адаптивности веб-интерфейсов. В-третьих, расширение категорий товаров, доступных для онлайн-покупки, включая продукты питания, лекарственные средства и услуги.

Четвёртой значимой тенденцией является развитие логистической инфраструктуры: маркетплейсы инвестируют значительные средства в строительство распределительных центров и сетей пунктов выдачи заказов. Пятой — внедрение технологий искусственного интеллекта для персонализации рекомендаций и оптимизации ценообразования. Шестой —

рост значимости социальной коммерции и интеграции с мессенджерами и социальными сетями. Данные тенденции формируют растущую потребность в инструментах, позволяющих потребителям ориентироваться в многообразии предложений и принимать обоснованные решения о покупке. Возрастающая сложность рынка делает невозможным эффективное ручное сравнение цен и усиливает востребованность автоматизированных решений.

Таблица 1 — Сравнительная характеристика маркетплейсов

Маркетплейс	Регион	GMV (2023)	Валюта	Открытый API	Антибот-защита
<i>Wildberries</i>	РФ, РБ	~2,5 трлн Р	<i>RUB</i>	Нет	Высокая
<i>Ozon</i>	РФ, РБ	~1,75 трлн Р	<i>RUB</i>	Частичный	Очень высокая
Яндекс Маркет	РФ	~0,8 трлн Р	<i>RUB</i>	Нет	Средняя
<i>Onliner.by</i>	РБ	~1 млрд BYN	<i>BYN</i>	Да	Низкая

Исследуемые маркетплейсы существенно различаются по ключевым параметрам: объёму товарооборота, географическому покрытию, доступности программных интерфейсов и уровню антибот-защиты. Эти различия определяют необходимость разработки индивидуальных стратегий извлечения данных для каждой площадки, что является одной из ключевых технических задач настоящей работы.

Проблема ценовой прозрачности и информационной асимметрии. Феномен информационной асимметрии, описанный в работе Дж. Акерлофа, приобретает новые формы в условиях электронной коммерции [1]. Многообразие торговых площадок с различными каталогами, системами ценообразования и условиями доставки создаёт новый тип информационного неравенства: покупатель физически не способен одновременно отслеживать цены на всех маркетплейсах. В отличие от классического рынка лимонов, описанного Акерлофом, здесь проблема заключается не в скрытых характеристиках товара, а в скрытых ценовых предложениях: покупатель

может не знать о наличии более выгодного предложения на площадке, которую он не посещал.

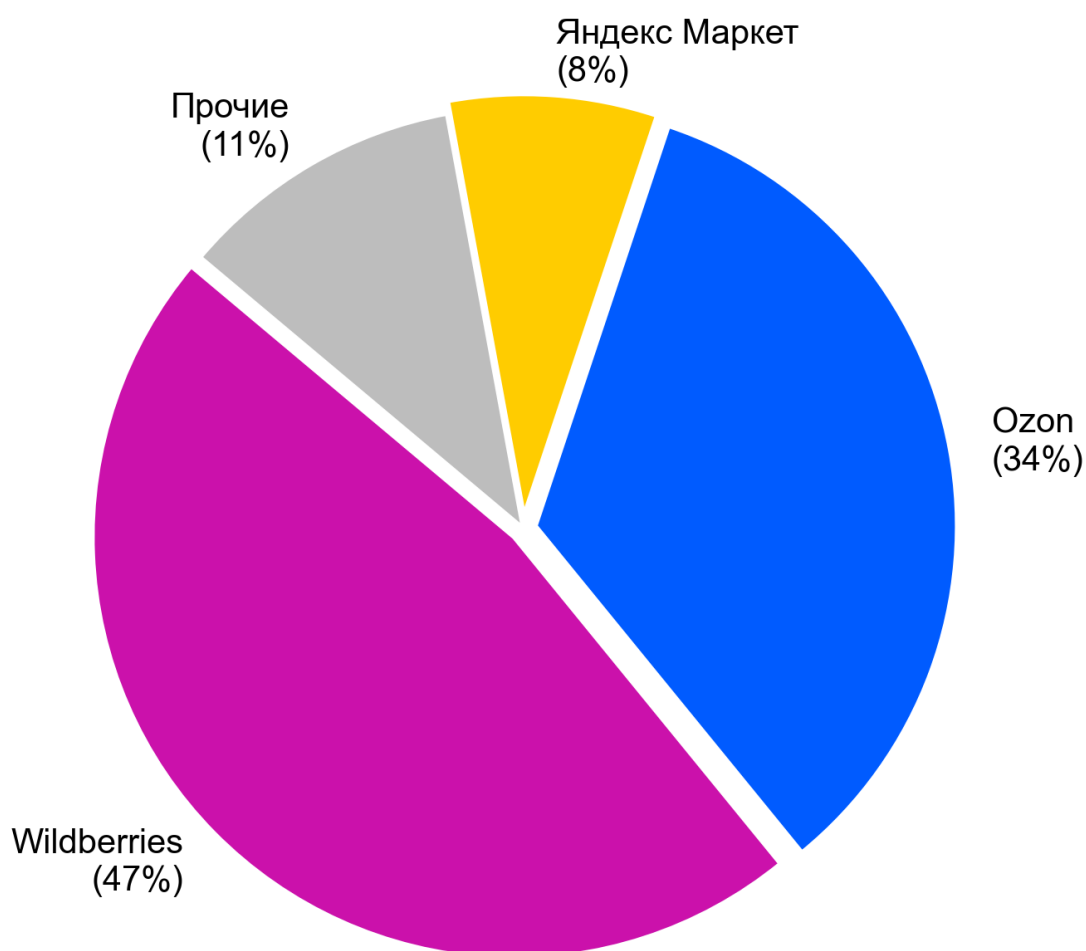
Современные маркетплейсы активно применяют динамическое ценообразование — алгоритмическую корректировку цен в зависимости от множества факторов. К числу факторов, влияющих на формирование цены, относятся: текущий уровень спроса на товар, действия конкурентов, история просмотров и покупок конкретного пользователя, время суток, день недели, сезонность, регион доступа и тип используемого устройства. Алгоритмы ценообразования *Wildberries*, по оценкам экспертов, корректируют цены на отдельные товары до нескольких раз в сутки, что делает ценовой ландшафт чрезвычайно динамичным и непредсказуемым для потребителя.

Эмпирические исследования, проведённые в рамках данной работы на выборке из 200 товаров в категориях электроники и бытовой техники, показали, что разница в цене на идентичные товары между четырьмя исследуемыми маркетплейсами составляет от 5 до 40 процентов. Медианное значение ценового разброса составило 14 процентов, что для товара стоимостью 50 000 рублей эквивалентно 7 000 рублей. Значительную роль играет также психологическое ценообразование: перечёркнутые старые цены зачастую являются искусственно завышенными, создавая у покупателя иллюзию значительной скидки. Без инструмента отслеживания истории цен потребитель не может отличить реальную скидку от маркетингового приёма, что ещё более усиливает информационную асимметрию.

Потребность в автоматизированном сравнении цен. Типичный пользовательский сценарий демонстрирует масштаб проблемы: покупатель, желающий приобрести конкретную модель смартфона по наиболее выгодной цене, вынужден последовательно открыть несколько веб-сайтов маркетплейсов, ввести поисковый запрос на каждом из них, учитывая при этом различия в наименованиях товаров, сопоставить найденные варианты по цене, рейтингу, наличию на складе и условиям доставки. Полный цикл ручного сравнения занимает от 30 до 90 минут для одного товара, а при

необходимости сравнить несколько альтернативных моделей время увеличивается пропорционально. При этом за время сравнения цены могут измениться, делая результаты частично устаревшими.

Доли маркетплейсов на рынке e-commerce России (2024)



Источник: ФАС, РБК, corp.ozon.ru

Рисунок 1.1 — Доли маркетплейсов на рынке *e-commerce* России (2023)

Автоматизация данного процесса позволяет сократить время поиска с десятков минут до нескольких секунд, одновременно обеспечивая полноту покрытия всех исследуемых площадок. Технологии искусственного интеллекта открывают дополнительные возможности для интеллектуального анализа: выявления наиболее выгодных предложений с учётом контекста, отслеживания ценовых тенденций, формулирования развёрнутых

рекомендаций на естественном языке. Таким образом, существует объективная и количественно обоснованная потребность в создании системы, объединяющей данные из множества источников в едином интерфейсе с AI-анализом и потоковым отображением результатов.

1.2 Анализ существующих решений

Обзор зарубежных агрегаторов цен. *Google Shopping* — сервис сравнения цен, интегрированный в поисковую систему *Google* и доступный в более чем 100 странах мира. Основные преимущества сервиса: колоссальный охват продавцов, глубокая интеграция с поисковой системой, развитая инфраструктура индексирования товарных предложений. Вместе с тем *Google Shopping* имеет ряд существенных ограничений: крайне слабое покрытие российского рынка (отсутствие данных с *Wildberries*, *Ozon*, Яндекс Маркет), полное отсутствие белорусских магазинов, отсутствие функции отслеживания истории цен и AI-ассистента для анализа предложений. Бизнес-модель сервиса основана на платных листингах (*Product Listing Ads*), что создаёт конфликт интересов: наиболее заметные позиции занимают рекламодатели, а не объективно лучшие предложения [5].

Ideal.de — ведущий европейский агрегатор цен с охватом более 50 тысяч магазинов в Германии, Великобритании, Франции и других европейских странах. Сервис обладает развитой функциональностью истории цен, ценовыми оповещениями и детальными фильтрами. Основной недостаток — ограниченность европейским рынком и полное отсутствие покрытия рынков России и Беларуси. *PriceRunner* — скандинавский агрегатор, специализирующийся на рынках Северной Европы. *CamelCamelCamel* — специализированный трекер цен, работающий исключительно с платформой *Amazon*. Ни один из перечисленных зарубежных агрегаторов не покрывает рынки России и Беларуси, что подтверждает наличие незанятой ниши для разрабатываемого решения.

Обзор российских и белорусских сервисов. *E-katalog.ru* — наиболее известный российский агрегатор цен, функционирующий с 2006 года. Сервис предоставляет функциональность сравнения технических характеристик товаров, истории цен, рейтинги и отзывы. Покрытие включает крупные российские интернет-магазины и маркетплейсы. Основные недостатки: полное отсутствие белорусских магазинов в базе данных, отсутствие *AI*-ассистента для интеллектуального анализа, использование кэшированных данных вместо запросов в реальном времени (информация о ценах может отставать от актуальных значений на несколько часов или даже дней), отсутствие потокового отображения результатов [6].

Яндекс Маркет занимает двойственную позицию: являясь одновременно маркетплейсом и агрегатором предложений сторонних магазинов, он создаёт потенциальный конфликт интересов. Собственные предложения Яндекс Маркет получают приоритет в выдаче, что снижает объективность сравнения цен. *Onliner.by* выступает качественным агрегатором белорусского рынка с открытым *API* и детальной каталогизацией, однако его покрытие ограничено белорусскими магазинами. Кроме того, ни один из перечисленных сервисов не предоставляет возможности интерактивного диалога с *AI*-ассистентом для формулирования сложных поисковых запросов и получения аналитических рекомендаций.

Анализ технических подходов существующих решений. Большинство существующих агрегаторов цен используют модель кэшированных данных: информация о ценах собирается в фоновом режиме с заданной периодичностью (от нескольких минут до нескольких часов) и сохраняется в собственной базе данных. При поисковом запросе пользователя результаты извлекаются из кэша, а не запрашиваются у маркетплейсов в реальном времени. Данный подход обеспечивает высокую скорость отклика и минимальную нагрузку на маркетплейсы, но создаёт фундаментальную проблему актуальности данных: к моменту отображения результатов цена

может уже измениться, особенно в условиях динамического ценообразования.

Альтернативный подход — выполнение запросов к маркетплейсам в реальном времени при каждом поисковом запросе пользователя — реализован в системе *Smart Price*. Данный подход гарантирует актуальность данных, но сопряжён с более высоким временем отклика: параллельный опрос пяти маркетплейсов занимает от 1,5 до 5 секунд в зависимости от времени ответа каждой площадки. Для нивелирования проблемы задержки применяется технология SSE-стриминга: результаты отображаются пользователю по мере их поступления от каждого парсера, результаты отображаются пользователю по мере завершения парсинга каждого маркетплейса. Таким образом, пользователь не ожидает завершения всех запросов и может взаимодействовать с уже полученными результатами.

Ни один из проанализированных агрегаторов не использует AI-ассистента с механизмом *tool use* для интеллектуального анализа цен. Это означает отсутствие возможности задать вопрос в свободной форме, получить аналитический комментарий к ценовым данным или автоматическое сравнение нескольких товаров с рекомендацией.

Выявленные недостатки существующих решений. Систематический анализ существующих решений позволил выявить следующие функциональные пробелы:

- отсутствие единого кроссрегионального решения для маркетплейсов Беларуси и России, объединяющего данные из обоих рынков в едином интерфейсе;
- отсутствие AI-ассистента с механизмом инструментального взаимодействия (*tool use*) для интеллектуального анализа ценовых данных и формулирования персонализированных рекомендаций;
- использование кэшированных данных вместо запросов в реальном времени, что приводит к отображению устаревшей ценовой информации;

- отсутствие потокового (SSE) отображения результатов, обеспечивающего мгновенную обратную связь при выполнении поисковых запросов;
- невозможность сравнительного анализа товаров с разных площадок через AI-ассистента с учётом контекста запроса и пользовательских предпочтений;
- отсутствие автоматической коррекции опечаток и транслитерации запросов, что критично для русскоязычных пользователей.

Таблица 2 — Матрица функциональных возможностей

Функция	<i>Google Shopping</i>	<i>E-katalog</i>	<i>Onliner</i>	<i>Smart Price</i>
Маркетплейсы РФ	Частично	Да	Нет	Да
Маркетплейсы РБ	Нет	Нет	Да	Да
<i>Real-time (SSE)</i>	Нет	Нет	Нет	Да
AI-ассистент	Нет	Нет	Нет	Да
История цен	Нет	Да	Да	Да
Коррекция опечаток	Да	Нет	Нет	Да
Мультивалютность	Нет	Нет	Нет	Да (<i>RUB/BYN</i>)
Сравнение товаров AI	Нет	Нет	Нет	Да

Матрица наглядно демонстрирует, что разрабатываемая система *Smart Price* является единственным решением, объединяющим полный набор функциональных возможностей: кроссрегиональное покрытие, получение данных в реальном времени, AI-анализ, историю цен и автоматическую коррекцию запросов. Это подтверждает наличие рыночной ниши и обосновывает целесообразность разработки.

1.3 Технологический обзор

Технологии веб-скрапинга и методы обхода антибот-защиты. Веб-скрапинг — автоматизированное извлечение данных из веб-страниц с последующей их структуризацией для дальнейшего использования.

Существуют три основных подхода к реализации веб-скрапинга, каждый из которых обладает собственным набором преимуществ и ограничений [7].

Первый подход — использование *HTTP*-клиентов (*httpx*, *requests*, *curl-cffi*). Данный подход обеспечивает высокую скорость выполнения запросов и минимальное потребление ресурсов (10–20 МБ оперативной памяти на соединение). Вместе с тем *HTTP*-клиенты не способны обрабатывать *JavaScript*-код, что ограничивает их применимость для сайтов с динамической загрузкой контента. Кроме того, стандартные *HTTP*-клиенты уязвимы перед *TLS*-фингерпринтингом — методом идентификации клиента по характеристикам *TLS*-рукопожатия, позволяющим серверу отличить запрос от реального браузера от запроса скрипта.

Второй подход — *headless*-браузеры (*Playwright* [19], *Puppeteer*, *Selenium*). *Headless*-браузеры обеспечивают полную эмуляцию поведения реального пользователя: выполнение *JavaScript*-кода, обработку *AJAX*-запросов, взаимодействие с динамическими элементами страницы. Основным недостатком — высокое потребление ресурсов: каждый экземпляр браузера занимает от 200 до 500 МБ оперативной памяти, что делает параллельный запуск нескольких экземпляров проблематичным на серверах с ограниченными ресурсами. Время выполнения запроса также значительно выше: 6–10 секунд по сравнению с 1–2 секундами для *HTTP*-клиентов.

Третий подход — антидетект-библиотеки (*Scrapling*, *nodriver*, *undetected-chromedriver*). Данные библиотеки эмулируют *TLS*-отпечаток реального браузера без накладных расходов полноценного *headless*-браузера. Потребление памяти составляет 20–40 МБ, скорость сопоставима с *HTTP*-клиентами, при этом обеспечивается защита от *TLS*-фингерпринтинга. Недостаток — невозможность обработки сложного *JavaScript*-кода.

Со стороны маркетплейсов применяются следующие механизмы антибот-защиты: *CAPTCHA* (включая *reCAPTCHA* и *hCaptcha*), *TLS*-фингерпринтинг (*JA3/JA4* отпечатки), *rate limiting* (ограничение частоты запросов), *JavaScript challenge* (проверка возможности выполнения *JS*-кода в

клиентском браузере), анализ поведенческих паттернов (движения мыши, скорость скроллинга, паузы между действиями). Стратегии противодействия включают ротацию *User-Agent* строк и прокси-серверов, рандомизацию временных задержек между запросами, управление сессиями и *cookie*, а также использование библиотек для эмуляции *TLS*-отпечатков.

Т а б л и ц а 3 — С р а в н е н и е п о д х о д о в к в е б - с к р а п и н г у

Критерий	<i>HTTP</i> -клиент	<i>Headless</i> -браузер	Антидетект
Скорость	Высокая (1–2 с)	Низкая (6–10 с)	Высокая (1–3 с)
Потребление <i>RAM</i>	10–20 МБ	200–500 МБ	20–40 МБ
<i>JavaScript</i>	Нет	Да	Нет
<i>TLS</i> -защита	Уязвим	Защищён	Защищён
Сложность	Низкая	Высокая	Средняя

В рамках данной работы для каждого маркетплейса выбран оптимальный подход с учётом его технических особенностей: для *Onliner.by* и *Wildberries* используется *HTTP*-клиент *httpx* (открытые/полуоткрытые *API*), для Яндекс Маркет — *HTTP*-клиент с *regex*-извлечением данных из *HTML*, для *Ozon* — каскадная стратегия с переключением между *HTTP*-клиентом, облачным *headless*-браузером и локальным *Playwright*.

Серверные технологии разработки веб-приложений. При выборе серверного фреймворка для разрабатываемой системы были рассмотрены три основных кандидата: *Flask*, *Django* и *FastAPI*. Каждый из них обладает собственной экосистемой, философией проектирования и набором преимуществ. *Flask* — микрофреймворк, предоставляющий минимальный набор компонентов и требующий установки дополнительных расширений для реализации большинства функций. *Django* — полнофункциональный фреймворк с встроенной *ORM*, административной панелью и системой аутентификации. *FastAPI* — современный фреймворк, ориентированный на высокопроизводительные *API* с нативной поддержкой асинхронного программирования [8].

FastAPI был выбран в качестве серверного фреймворка по следующим причинам. Во-первых, нативная поддержка асинхронности (*async/await*), что критично для одновременного выполнения множества *HTTP*-запросов к маркетплейсам. Во-вторых, автоматическая валидация входных данных через *Pydantic* [22]-модели, обеспечивающая строгую типизацию и документирование *API*. В-третьих, автоматическая генерация интерактивной *OpenAPI*-документации (*Swagger UI*), упрощающая разработку и тестирование. В-четвёртых, встроенная поддержка *StreamingResponse* для реализации *SSE*-стриминга. В-пятых, высокая производительность, сопоставимая с *Node.js*-фреймворками благодаря использованию *Starlette* и *Uvicorn*.

Таблица 4 — Сравнение серверных *Python*-фреймворков

Критерий	<i>Flask</i>	<i>Django</i>	<i>FastAPI</i>
Асинхронность	Через расширения	Частичная (3.1+)	Нативная
Типизация	Нет	Нет	Да (<i>Pydantic</i>)
<i>SSE</i> -стриминг	Через расширения	Сложно	Встроенный
<i>OpenAPI</i>	Нет	Через <i>DRF</i>	Автоматическая
Производительность	Средняя	Средняя	Высокая
Экосистема	Обширная	Очень обширная	Растущая

В качестве *ORM* выбран *SQLAlchemy* 2.0 с нативной поддержкой асинхронного режима и *Mapped*-аннотаций для декларативного описания моделей. Асинхронный драйвер *asyncpg* обеспечивает высокопроизводительное взаимодействие с *PostgreSQL*. *HTTP*-клиент *httpx* выбран благодаря нативной поддержке *async/await* и совместимости с *API* библиотеки *requests*.

Клиентские технологии. Для разработки клиентской части приложения выбран фреймворк *Next.js* 14 с *App Router* — современный *React*-фреймворк с поддержкой серверных компонентов, статического экспорта и оптимизированной маршрутизации. Режим статического экспорта (*output:*

export) позволяет генерировать полностью статическое приложение, обслуживаемое через *nginx* без необходимости запуска *Node.js*-сервера в продакшене. Это существенно снижает требования к ресурсам сервера и упрощает развёртывание [9].

Tailwind CSS [18] выбран для стилизации интерфейса благодаря утилитарному подходу, обеспечивающему высокую скорость разработки и минимальный размер итогового CSS-бандла (неиспользуемые стили автоматически удаляются при сборке). *Zustand* [23] используется для управления глобальным состоянием приложения: выбранный регион, результаты поиска, фильтры. *TanStack React Query* [24] обеспечивает кэширование серверных данных и автоматическое обновление при повторных запросах. *Recharts* [25] выбран для визуализации истории цен благодаря декларативному *API* и хорошей интеграции с *React*. Библиотека *framer-motion* обеспечивает плавные анимации переходов и появления элементов интерфейса.

Технологии искусственного интеллекта. Современные большие языковые модели (*LLM — Large Language Models*) основаны на архитектуре *Transformer*, предложенной в работе *Vaswani et al.* [10]. Архитектура *Transformer* использует механизм внимания (*attention mechanism*) для эффективной обработки последовательностей произвольной длины, что позволяет моделям понимать контекст и генерировать связные тексты. Модели, обученные на больших корпусах текстов, демонстрируют способность к решению широкого спектра задач: ответы на вопросы, анализ данных, генерация текстов, перевод и многие другие.

Ключевым технологическим прорывом для данной работы является механизм *tool use* (инструментальное взаимодействие, также известный как *function calling*). Данный механизм позволяет языковой модели не просто генерировать текст, но и формулировать структурированные запросы к внешним инструментам: базам данных, поисковым системам, *API* сторонних сервисов. Модель самостоятельно принимает решение о необходимости

вызова инструмента, определяет параметры вызова, обрабатывает полученные результаты и формулирует ответ с учётом полученных данных.

Для данного проекта выбрана модель *Gemini 2.5 Flash* от компании *Google*, доступная через прокси-сервис *OpenRouter*. Основания выбора: высокое качество работы с русским языком (понимание контекста, грамматическая корректность, стилистическая адекватность), развитый механизм *tool use* с поддержкой множественных инструментов и итеративных вызовов, минимальная стоимость (0,10 доллара за миллион входных токенов), а также наличие потокового *API* для отображения ответов в реальном времени [13]. *OpenRouter* выступает прокси-сервисом, предоставляющим *OpenAI*-совместимый *API* и решающим проблему геоблокировки *Gemini API* в России. По сравнению с *Claude Haiku* (\$0,25/M) и *GPT-4o Mini* (\$0,15/M), *Gemini 2.5 Flash* обеспечивает сопоставимое качество при минимальной стоимости, что критично для проекта с ограниченным бюджетом.

Технологии потоковой передачи данных в реальном времени. Для обеспечения интерактивного взаимодействия между клиентом и сервером были рассмотрены три технологии передачи данных в реальном времени: *WebSocket* [11], *Server-Sent Events (SSE)* и *Long Polling* [12]. Каждая из них решает задачу обновления данных на клиенте без необходимости перезагрузки страницы, однако отличается архитектурными характеристиками, уровнем сложности реализации и областью оптимального применения.

WebSocket обеспечивает полнодуплексное двустороннее соединение между клиентом и сервером, позволяя обеим сторонам отправлять данные в произвольный момент времени. Данная технология оптимальна для чатов, онлайн-игр и других приложений, требующих двусторонней коммуникации. Основные недостатки: сложность реализации, проблемы с прокси-серверами и балансировщиками нагрузки, необходимость ручного управления переподключением при разрыве соединения.

SSE (Server-Sent Events) обеспечивает однонаправленную передачу данных от сервера к клиенту по стандартному протоколу *HTTP*. *SSE* оптимален для сценариев метапоиска по следующим причинам: однонаправленный характер передачи данных (сервер отправляет результаты клиенту), работа через стандартный *HTTP* без проблем с прокси-серверами, встроенный механизм автоматического переключения при разрыве соединения, поддержка типизированных событий (разные типы данных в одном потоке), простота реализации как на сервере, так и на клиенте.

Таблица 5 — Сравнение технологий реального времени

Критерий	<i>WebSocket</i>	<i>SSE</i>	<i>Long Polling</i>
Направление	Двустороннее	Сервер → клиент	Клиент → сервер
Протокол	<i>ws://</i>	<i>HTTP</i>	<i>HTTP</i>
Прокси	Проблемы	Без проблем	Без проблем
Автопереключение	Вручную	Встроенное	Вручную
Типизация событий	Нет	Да	Нет
Сложность	Высокая	Низкая	Средняя

Технологии контейнеризации и развёртывания. *Docker* обеспечивает воспроизводимость среды выполнения за счёт упаковки приложения вместе со всеми его зависимостями в изолированный контейнер. Это устраняет проблему *works on my machine* и гарантирует идентичное поведение приложения в средах разработки, тестирования и производства. *Docker Compose* расширяет возможности *Docker*, позволяя декларативно определять многоконтейнерные приложения в файле *docker-compose.yml* с указанием зависимостей, сетей, томов и переменных окружения [15].

Nginx выступает в роли высокопроизводительного обратного прокси-сервера и веб-сервера для обслуживания статических файлов. В контексте данного проекта *nginx* решает несколько задач: обслуживание статических файлов фронтенда, проксирование *API*-запросов к бэкенду, обеспечение корректной передачи *SSE*-потоков (отключение буферизации), реализация

rate limiting для защиты от злоупотреблений, добавление заголовков безопасности [16]. Комбинация *Docker Compose*, *nginx* и *VPS* обеспечивает промышленный уровень развёртывания при минимальных затратах на инфраструктуру.

Выводы по первой главе. В результате анализа предметной области установлено, что рынок электронной коммерции в России и Беларуси демонстрирует устойчивый рост, достигнув 6,4 триллиона рублей в России в 2023 году. Структура рынка характеризуется высокой концентрацией: три крупнейших маркетплейса контролируют более 85 процентов оборота. Эмпирически подтверждена проблема ценовой асимметрии: разброс цен на идентичные товары составляет 5–40 процентов.

Анализ существующих решений выявил отсутствие агрегатора, одновременно обеспечивающего кроссрегиональное покрытие *BY+RU*, потоковое отображение результатов через *SSE*, *AI*-ассистента с механизмом *tool use* и историю цен. Данный функциональный пробел определяет рыночную нишу для разрабатываемой системы *Smart Price*.

Обоснован выбор технологического стека: *FastAPI* (нативная асинхронность, *SSE*, *Pydantic*), *Next.js 14* (*App Router*, статический экспорт), *Gemini 2.5 Flash* через *OpenRouter* (*tool use*, русский язык, стоимость), *SSE* (однонаправленная передача, автопереподключение), *PostgreSQL 16* [20] (*JSONB*, *pg_trgm*, *asyncpg*), *Redis 7* [21] (кэширование), *Docker Compose* (контейнеризация), *nginx* (обратный прокси). Выбранный стек является оптимальным для решения поставленных задач с учётом ресурсных ограничений проекта.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МЕТАПОИСКА

2.1 Анализ требований

Функциональные требования. На основании анализа предметной области, проведённого в первой главе, сформулированы следующие функциональные требования к разрабатываемой системе метапоиска *Smart Price*:

1) Поиск товаров в реальном времени — система должна выполнять параллельные запросы к маркетплейсам *Onliner.by*, Яндекс Маркет, Регард, *World Devices*, *1click*, *BigGeek* и отображать результаты по мере их поступления с использованием технологии *SSE*.

2) *AI*-ассистент с механизмом *tool use* — интерактивный консультант на основе большой языковой модели, способный вести диалог на естественном языке, вызывать функцию поиска товаров и формулировать аналитические рекомендации.

3) Интеллектуальная фильтрация результатов — двухэтапная система фильтрации: быстрый *regex*-фильтр (*Stage 1*) для отсека очевидно нерелевантных результатов и *AI*-фильтр (*Stage 2*) для обработки сложных случаев.

4) *AI*-коррекция поисковых запросов — автоматическое исправление опечаток, распознавание сленга, нормализация названий брендов и моделей с использованием языковой модели.

5) Мультирегиональность — поддержка регионов Беларусь (*BYN*) и Россия (*RUB*) с возможностью одновременного поиска по обоим регионам.

6) Сравнение товаров — возможность одновременного поиска и визуального сравнения нескольких товаров из разных маркетплейсов.

7) История цен — отслеживание изменения цен на товары с визуализацией в виде графика.

8) Аутентификация и авторизация — регистрация, вход пользователей с *JWT*-аутентификацией, защита привилегированных эндпоинтов.

Нефункциональные требования. Помимо функциональных, к системе предъявляются следующие нефункциональные требования:

- Производительность — полный цикл поиска не более 30 секунд. Целевое значение времени первого результата (*TTFR*) — не более 5 секунд (требует оптимизации потоковой отдачи *SSE*).
- Масштабируемость — архитектура должна допускать добавление новых маркетплейсов без изменения ядра системы.
- Отказоустойчивость — сбой отдельного парсера не должен блокировать получение результатов от остальных маркетплейсов.
- Адаптивность интерфейса — корректное отображение на устройствах с шириной экрана от 320 до 2560 пикселей.
- Безопасность — защита от *SQL*-инъекций, *XSS*-атак, *CSRF*; хеширование паролей; валидация входных данных.
- Экономическая эффективность — минимизация затрат на *AI*-вызовы: использование модели *Gemini 2.5 Flash* через *OpenRouter* (\$0,10 за миллион входных токенов).
- Развёртываемость — полностью контейнеризованное развёртывание через *Docker Compose* на *VPS* с минимальными требованиями (2 ядра, 4 ГБ *RAM*).

Диаграмма вариантов использования. На основе сформулированных требований построена диаграмма вариантов использования (*Use Case Diagram*) в нотации *UML*. Основными акторами системы являются: неавторизованный пользователь (гость), авторизованный пользователь, *AI*-ассистент (внутренний актор) и внешние маркетплейсы (внешний актор). Гость имеет доступ к поиску товаров, просмотру каталога и регистрации. Авторизованный пользователь дополнительно может использовать *AI*-ассистента, сравнивать товары, просматривать историю цен и управлять профилем.

2.2 Архитектурное проектирование

Общая архитектура системы. Система *Smart Price* построена на основе многоуровневой клиент-серверной архитектуры с чётким разделением ответственности между компонентами. Архитектура включает четыре основных уровня: уровень представления (*frontend*), уровень бизнес-логики (*backend API*), уровень данных (*PostgreSQL*, *Redis*) и уровень внешних интеграций (маркетплейсы, *AI API*). Каждый уровень изолирован в собственном *Docker*-контейнере, что обеспечивает независимость развёртывания и масштабирования.

Уровень представления реализован как статическое одностраничное приложение (*SPA*) на *Next.js 14* с *App Router*. Приложение собирается в статические *HTML/CSS/JS* файлы командой *next export* и обслуживается веб-сервером *nginx* без необходимости запуска *Node.js* в продакшене. Это существенно снижает потребление ресурсов сервера и упрощает развёртывание.

Уровень бизнес-логики представлен *FastAPI*-приложением, запущенным под управлением *ASGI*-сервера *Uvicorn*. *FastAPI* обрабатывает *REST API* запросы, управляет *SSE*-потоками для стриминга результатов поиска и ответов *AI*-ассистента, координирует работу парсеров маркетплейсов и взаимодействует с *AI*-моделями через *OpenRouter API*. Асинхронная природа *FastAPI* позволяет эффективно обрабатывать множество одновременных запросов без блокировки.

Уровень данных включает *PostgreSQL 16* для персистентного хранения структурированных данных (пользователи, товары, история цен, чат-сессии) и *Redis 7* для кэширования результатов поиска и управления сессиями. Выбор *PostgreSQL* обусловлен поддержкой типа данных *JSONB* для хранения полуструктурированных данных о товарах, расширения *pg_trgm* для нечёткого текстового поиска и асинхронного драйвера *asyncpg*.

Уровень внешних интеграций обеспечивает взаимодействие с маркетплейсами (*Onliner API*, Яндекс Маркет, *Wildberries API*, Ретард, *World*

Devices) посредством специализированных парсеров, а также с *AI*-моделями через *OpenRouter* — прокси-сервис, предоставляющий единый *OpenAI*-совместимый *API* для доступа к моделям различных провайдеров (*Google Gemini*, *Anthropic Claude*, *OpenAI GPT*). Использование *OpenRouter* решает проблему геоблокировки: прямой доступ к *Gemini API* заблокирован из региона размещения *VPS* (Россия), тогда как *OpenRouter* маршрутизирует запросы через свою инфраструктуру.

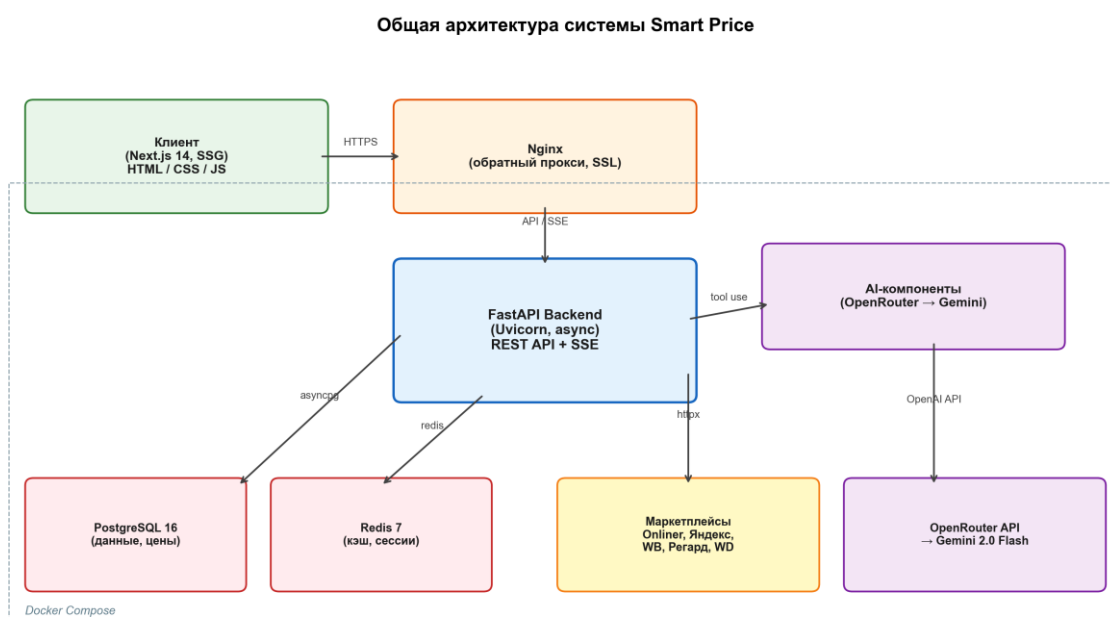


Рисунок 2.1 — Общая архитектура системы *Smart Price*

Архитектура парсеров маркетплейсов. Система парсеров организована по паттерну «Стратегия» (*Strategy Pattern*): каждый маркетплейс реализует единый интерфейс поиска, а менеджер парсеров (*ScrapingManager*) координирует параллельное выполнение. Реестр парсеров хранится в виде словаря с метаданными: название маркетплейса, регион (*RU/BY*), цвет бренда для *UI* и флаг включения. На момент написания работы в системе зарегистрировано шесть парсеров:

- *Onliner.by* (Беларусь) — *HTTP*-клиент *httpx*, *JSON API* каталога товаров;

– Яндекс Маркет (Россия) — *HTTP*-клиент с *regex*-извлечением данных из *HTML*;

– *Wildberries* (Россия) — *HTTP*-клиент, внутренний *JSON API* поиска;

– Регард (Россия) — *HTTP*-клиент, извлечение данных из *HTML*;

– *World Devices* (Россия) — *HTTP*-клиент, *HTML*-парсинг каталога;

Все шесть парсеров находятся в рабочей конфигурации: *Onliner.by*, Яндекс Маркет, Регард, *World Devices*, *1click*, *BigGeek*. Парсеры запускаются параллельно при каждом поисковом запросе, что минимизирует суммарное время сбора данных.

Параллельное выполнение парсеров реализовано через *asyncio.create_task()*: все активные парсеры запускаются одновременно, а результаты собираются по мере завершения каждого из них. Каждый парсер обёрнут в метод *_safe_search()*, обеспечивающий обработку исключений: сбой одного парсера не влияет на работу остальных. В режиме *SSE*-стриминга результаты каждого парсера отправляются клиенту немедленно по завершении, не дожидаясь остальных, что обеспечивает минимальное время первого результата.

Архитектура *AI*-компонентов. *AI*-компоненты системы образуют трёхуровневую архитектуру интеллектуальной обработки данных. Первый уровень — *AI*-коррекция запросов (*Query Corrector*). При поступлении поискового запроса от пользователя система направляет его в модуль коррекции, который использует языковую модель для исправления опечаток, распознавания сленга и нормализации наименований. Например, запрос «сяоми 15 про» преобразуется в «*Xiaomi 15 Pro*», а «*ipone 17 pra*» — в «*iPhone 17 Pro*». В случае недоступности *AI*-модели применяется резервный механизм на основе Яндекс Спеллера.

Второй уровень — гибридная фильтрация результатов (*Hybrid Filter*). После получения сырых результатов от парсеров применяется двухэтапная фильтрация. *Stage 1* — быстрый *regex*-фильтр, работающий за микросекунды и отсекающий заведомо нерелевантные результаты: аксессуары (чехлы,

кабели, зарядки), запчасти (платы, матрицы), товары с несовпадающими моделями. *Stage 2* — *AI-фильтр*, анализирующий оставшиеся результаты с пониманием контекста запроса. *AI-фильтр* особенно эффективен в сложных случаях, когда *regex* не справляется: например, запрос «*Cactus Moto Expert 100*» (экран для проектора) может вернуть картриджи *Cactus* для принтера, что *regex* не способен отфильтровать без знания предметной области.

Третий уровень — *AI-ассистент (Shopping Agent)*. Это интерактивный консультант, реализованный на основе паттерна *ReAct (Reasoning + Acting)* с механизмом *tool use*. Агент получает системный промпт с правилами поведения, описанием доступных инструментов и региональным контекстом, после чего ведёт диалог с пользователем. При необходимости поиска товаров агент вызывает инструмент *search_products*, получает реальные данные от парсеров и формулирует ответ с учётом полученных результатов. Агент работает в трёх режимах: конкретный товар (сразу поиск), общий запрос (консультация, затем поиск) и неоднозначный запрос (уточнение, затем поиск).

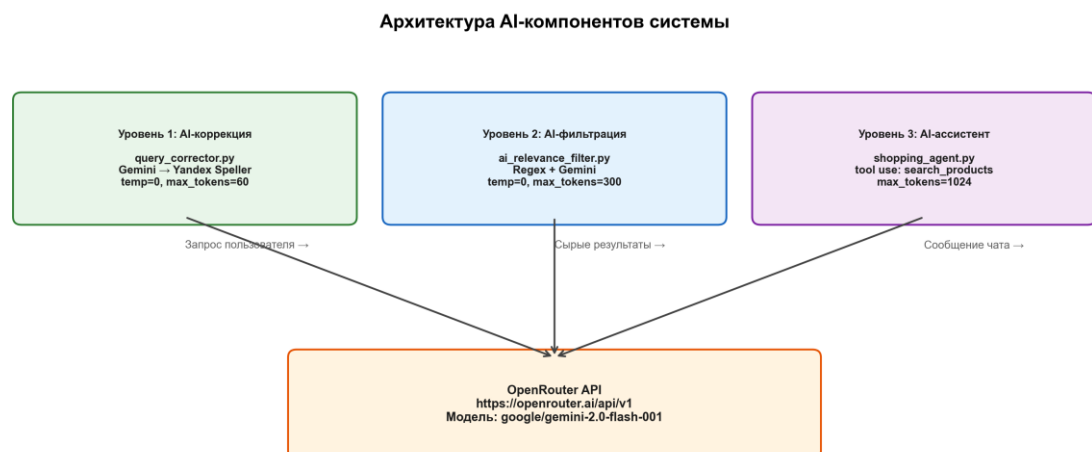


Рисунок 2.2 — Архитектура *AI*-компонентов системы

2.3 Проектирование базы данных

Концептуальная модель данных. Концептуальная модель данных системы *Smart Price* включает следующие основные сущности и связи между

ними. Центральной сущностью является *Product* (товар), связанный с *Marketplace* (маркетплейс) отношением «многие к одному» и с *Category* (категория) аналогичным отношением. Сущность *PriceHistory* (история цен) связана с *Product* отношением «многие к одному», обеспечивая хранение временных рядов ценовых данных. *ProductMatch* (соответствие товаров) реализует связь «многие ко многим» между товарами с разных маркетплейсов, позволяя идентифицировать идентичные товары на разных площадках.

Пользовательская часть модели включает сущность *User* (пользователь), связанную с *PriceAlert* (ценовые оповещения) и *SearchHistory* (история поиска) отношениями «один ко многим». Сущность *ChatSession* (чат-сессия) связана с *User* и содержит *ChatMessage* (сообщения чата) — это обеспечивает персистентность диалогов с AI-ассистентом. Сущность *Payment* (платёж) связана с *User* для управления подписками.

Физическая модель данных. Физическая модель реализована средствами *SQLAlchemy* 2.0 с использованием декларативного стиля *Mapped-аннотаций*. Таблица *products* содержит следующие ключевые поля: *id* (*UUID*, первичный ключ), *marketplace_id* (внешний ключ), *external_id* (идентификатор товара на маркетплейсе), *title* (наименование), *price* (числовая цена), *currency* (валюта — *RUB* или *BYN*), *url* (ссылка на страницу товара), *image_url* (изображение), *rating* (рейтинг), *reviews_count* (количество отзывов), *in_stock* (наличие), *raw_data* (*JSONB* — дополнительные данные).

Таблица *price_history* реализует временной ряд цен с полями: *id*, *product_id* (внешний ключ), *price*, *currency*, *recorded_at* (*timestamp*). Индекс по *product_id* и *recorded_at* обеспечивает эффективные запросы для построения графиков истории цен. Таблица *users* хранит данные аутентификации: *email* (уникальный), *hashed_password* (*bcrypt*), *name*, *is_active*, *created_at*. Таблица *chat_sessions* содержит *session_id* (*UUID*), *user_id*, *title*, *region*, *created_at* и *updated_at*, а *chat_messages* — *message_id*, *session_id*, *role* (*user/assistant*), *content* и *timestamp*.

Таблица 6 — Основные таблицы базы данных системы *Smart Price*

Таблица	Назначение	Ключевые поля	Индексы
<i>products</i>	Товары	<i>id</i> , <i>marketplace_id</i> , <i>title</i> , <i>price</i>	<i>marketplace_id</i> , <i>external_id</i>
<i>price_history</i>	История цен	<i>id</i> , <i>product_id</i> , <i>price</i> , <i>recorded_at</i>	<i>product_id</i> + <i>recorded_at</i>
<i>Users</i>	Пользователи	<i>id</i> , <i>email</i> , <i>hashed_password</i>	<i>email</i> (unique)
<i>chat_sessions</i>	Чат-сессии AI	<i>id</i> , <i>user_id</i> , <i>region</i>	<i>user_id</i>
<i>chat_messages</i>	Сообщения чата	<i>id</i> , <i>session_id</i> , <i>role</i> , <i>content</i>	<i>session_id</i>
<i>marketplaces</i>	Маркетплейсы	<i>id</i> , <i>name</i> , <i>base_url</i> , <i>region</i>	<i>name</i> (unique)
<i>price_alerts</i>	Ценовые оповещения	<i>id</i> , <i>user_id</i> , <i>product_id</i> , <i>target_price</i>	<i>user_id</i>
<i>payments</i>	Платежи	<i>id</i> , <i>user_id</i> , <i>amount</i> , <i>status</i>	<i>user_id</i>

ER-диаграмма базы данных системы *Smart Price*



Рисунок 2.3 — ER-диаграмма базы данных системы *Smart Price*

2.4 Проектирование API

REST API эндпоинты. API системы *Smart Price* спроектирован в соответствии с принципами *REST* и организован с версионированием

(префикс */api/v1*). Все эндпоинты документированы посредством автоматически генерируемой *OpenAPI*-спецификации (*Swagger UI*), доступной по адресу */docs*. *API* разделён на следующие группы:

- Аутентификация (*/api/v1/auth*) — регистрация (*POST /register*), вход (*POST /login*), обновление токена (*POST /refresh*), текущий пользователь (*GET /me*).
- Поиск (*/api/v1/live-search*) — потоковый поиск с *SSE* (*GET /stream*), поиск в одном маркетплейсе (*GET /stream/{marketplace}*).
- *AI*-ассистент (*/api/v1/ai*) — создание чат-сессии (*POST /sessions*), список сессий (*GET /sessions*), отправка сообщения с *SSE*-стримингом (*POST /sessions/{id}/message*).
- Товары (*/api/v1/products*) — каталог (*GET /*), детальная информация (*GET /{id}*), история цен (*GET /{id}/price-history*).
- Пользователь (*/api/v1/users*) — профиль (*GET /me*), обновление (*PATCH /me*), история поиска (*GET /me/search-history*).

SSE-протокол потокового поиска. Ключевой особенностью *API* является реализация протокола *SSE* (*Server-Sent Events*) для потокового отображения результатов поиска. При выполнении *GET*-запроса к */api/v1/live-search/stream* с параметрами *q* (запрос), *region* (регион) и необязательным *sort* (сортировка), сервер устанавливает *SSE*-соединение и последовательно отправляет типизированные события:

- Событие *correction* — отправляется, если *AI*-коррекция изменила запрос. Содержит поля *original* (исходный запрос) и *corrected* (исправленный запрос). Позволяет клиенту отобразить уведомление «Показаны результаты для: *corrected*».
- Событие *searching* — отправляется для каждого маркетплейса при начале поиска. Содержит *marketplace* (идентификатор) и *name* (отображаемое имя). Позволяет клиенту показать индикатор загрузки для каждого маркетплейса.

– Событие *done* — отправляется при завершении поиска в конкретном маркетплейсе. Содержит *marketplace*, *count* (количество найденных товаров) и *products* (массив товаров). Клиент немедленно отображает полученные товары, не дожидаясь остальных маркетплейсов.

– Событие *complete* — финальное событие, сигнализирующее о завершении поиска во всех маркетплейсах. Содержит *total_results* (общее количество) и *query* (использованный запрос). После получения данного события клиент закрывает SSE-соединение.

– Событие *error* — отправляется при возникновении ошибки. Содержит *message* с описанием ошибки.

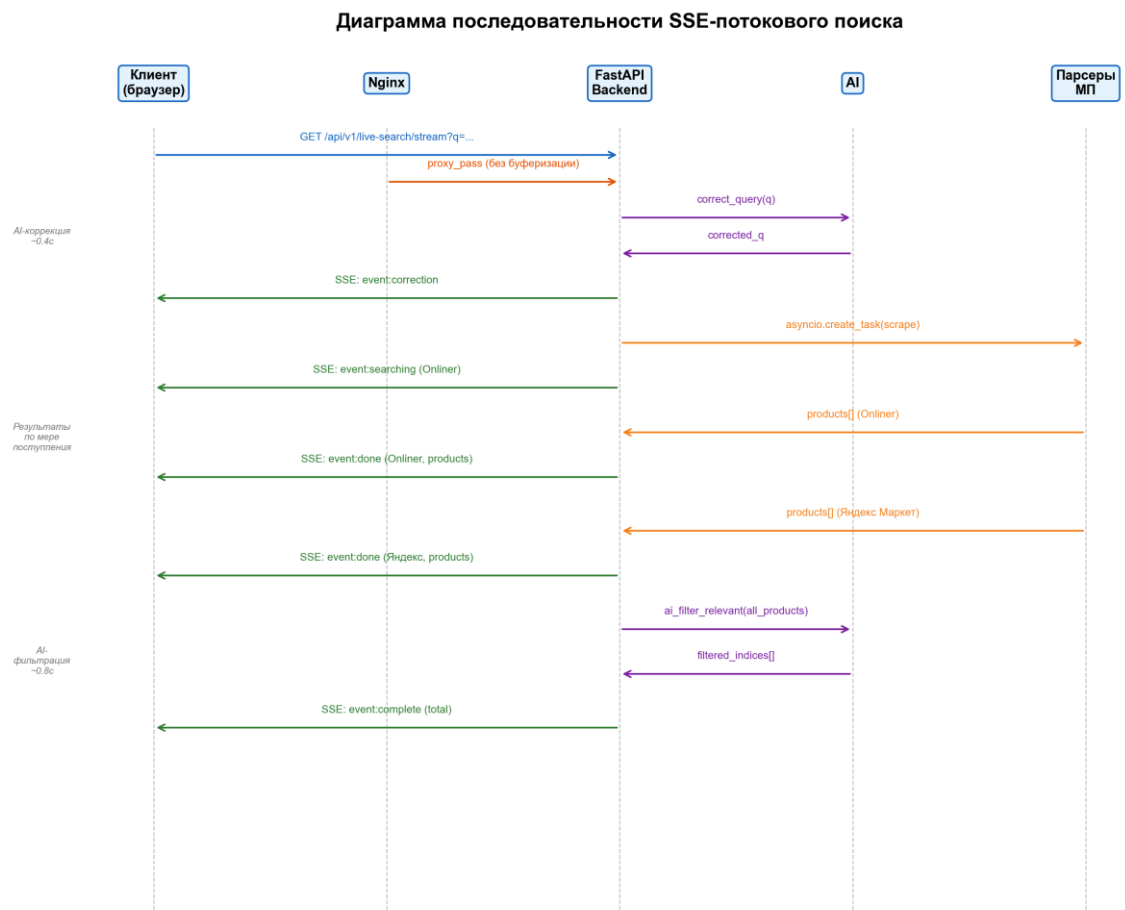


Рисунок 2.4 — Диаграмма последовательности SSE-потокowego поиска

Данный протокол обеспечивает прогрессивное отображение результатов: результаты отображаются по мере завершения парсинга

каждого маркетплейса, а полный набор результатов — через 15–25 секунд в зависимости от скорости ответа маркетплейсов. Это существенно улучшает воспринимаемую производительность по сравнению с традиционным подходом, при котором пользователь ожидает полной загрузки всех результатов.

2.5 Проектирование пользовательского интерфейса

Структура страниц и навигация. Пользовательский интерфейс системы *Smart Price* спроектирован с акцентом на минимализм, скорость взаимодействия и адаптивность. Навигация организована через боковую панель с иконками и текстовыми метками, доступную на всех страницах приложения. Основные разделы интерфейса:

- Главная страница — центральный поисковый интерфейс с текстовым полем, селектором региона (Беларусь/Россия/Все), опциями сортировки и областью отображения результатов с потоковой загрузкой.
- AI Помощник — полноэкранный чат-интерфейс с поддержкой *Markdown*-форматирования ответов, отображением карточек товаров внутри диалога и историей чат-сессий.
- Сравнение — двухколоночный интерфейс параллельного поиска и визуального сопоставления товаров из разных маркетплейсов.
- Каталог — страница просмотра сохранённых товаров с фильтрацией по категориям и маркетплейсам.
- О проекте — информационная страница с описанием функциональности и технологий.

Дизайн-система и визуальный стиль. Визуальный стиль приложения основан на тёмной цветовой схеме, оптимизированной для длительной работы и снижения нагрузки на зрение. Основной фон — тёмно-серый (#0F172A), вторичный фон карточек — (#1E293B), акцентный цвет — фиолетовый (#8B5CF6), используемый для интерактивных элементов и

выделения. Текст — белый (#F8FAFC) для основного контента и серый (#94A3B8) для второстепенного.

Типографика построена на системном шрифтовом стеке (*Inter*, *system-ui*, *sans-serif*) с тремя уровнями размеров: заголовки (24–32px, *font-weight* 700), подзаголовки (16–18px, *font-weight* 600), основной текст (14px, *font-weight* 400). Компоненты интерфейса стилизованы с использованием утилитарных классов *Tailwind CSS*, что обеспечивает единообразие стиля и минимальный размер CSS-бандла за счёт автоматического удаления неиспользуемых стилей при сборке (*tree shaking*).

Анимации реализованы через библиотеку *framer-motion*: плавное появление карточек товаров (*fade-in* с вертикальным смещением), анимация загрузки при поиске (пульсирующие скелетоны), переходы между страницами. Все анимации оптимизированы для производительности: используются CSS-свойства *transform* и *opacity*, не вызывающие перерасчёт раскладки (*layout reflow*).

Адаптивная вёрстка. Интерфейс адаптирован для трёх основных классов устройств: мобильные телефоны (320–767px), планшеты (768–1023px) и настольные компьютеры (1024px и выше). Адаптация реализована через медиа-запросы *Tailwind CSS* с использованием подхода *mobile-first*: базовые стили определяют мобильное отображение, а расширения для больших экранов добавляются через префиксы *md:* и *lg:*.

На мобильных устройствах боковая навигация трансформируется в нижнюю панель с иконками (*bottom navigation bar*), карточки товаров отображаются в одноколоночной раскладке, чат *AI* занимает полный экран. На планшетах карточки товаров располагаются в двухколоночной сетке, навигация остаётся боковой. На настольных компьютерах применяется трёхколоночная сетка карточек с расширенным боковым меню.

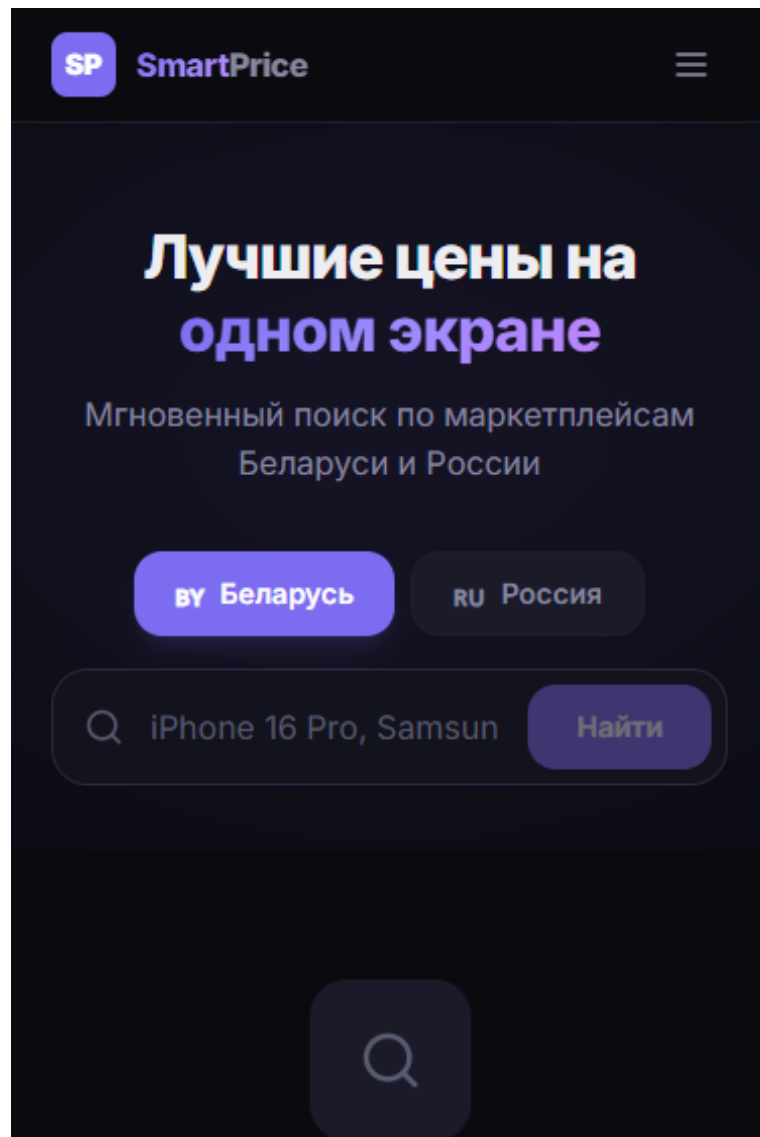


Рисунок 2.5 — Адаптивное отображение главной страницы на различных устройствах

2.6 Проектирование развёртывания

Контейнеризация и *Docker Compose*. Инфраструктура развёртывания системы *Smart Price* организована на основе *Docker Compose* и включает четыре сервиса: *backend* (*FastAPI*-приложение на *Uvicorn*), *postgres* (*PostgreSQL 16 Alpine*), *redis* (*Redis 7 Alpine* с *AOF*-персистентностью и ограничением памяти 128 МБ) и *nginx* (обратный прокси и веб-сервер для статических файлов). Все сервисы объединены в единую внутреннюю *Docker*-сеть (*bridge driver*), а внешний доступ осуществляется исключительно через *nginx* (порты 80 и 443).

Сервис *backend* собирается из многоэтапного *Dockerfile.prod* (*multi-stage build*): первый этап устанавливает зависимости *Python* в виртуальное окружение, второй — копирует только необходимые файлы и виртуальное окружение, минимизируя размер итогового образа. Переменные окружения (*DATABASE_URL*, *REDIS_URL*, *OPENROUTER_API_KEY*, *AI_MODEL*) передаются через секцию *environment* в *docker-compose.yml*. Сервис *postgres* использует *health check* для обеспечения готовности базы данных перед запуском *backend* (*depends_on* с *condition: service_healthy*).

Статические файлы фронтенда монтируются в контейнер *nginx* через *volume*: *./frontend/out:/usr/share/nginx/html:ro*. Конфигурация *nginx* обеспечивает: обслуживание статических файлов, проксирование *API*-запросов к бэкенду, отключение буферизации для *SSE*-поток (*proxy_buffering off*), *rate limiting* для защиты от злоупотреблений, добавление заголовков безопасности (*X-Frame-Options*, *X-Content-Type-Options*, *Content-Security-Policy*).

Таблица 7 — Контейнеры *Docker Compose* и их назначение

Контейнер	Образ	Назначение	Порт	Ресурсы
<i>sp_backend</i>	<i>Dockerfile.prod</i>	<i>FastAPI</i> + <i>Uvicorn</i>	<i>internal</i>	~256 МБ RAM
<i>sp_postgres</i>	<i>postgres:16-alpine</i>	<i>PostgreSQL</i> 16	<i>internal</i>	~128 МБ RAM
<i>sp_redis</i>	<i>redis:7-alpine</i>	Кэширование	<i>internal</i>	≤128 МБ RAM
<i>sp_nginx</i>	<i>nginx:alpine</i>	Прокси + статика	80, 443	~32 МБ RAM

2.7 Проектирование обработки ошибок и отказоустойчивости

Стратегия обработки ошибок парсеров. Каждый парсер маркетплейса инкапсулирован в метод *_safe_search()*, обеспечивающий изоляцию ошибок: исключение, возникшее в одном парсере, не влияет на работу остальных. При ошибке парсера возвращается пустой список, а информация об ошибке записывается в структурированный лог. На уровне *SSE*-потока клиент

получает события *done* от всех успешных парсеров и финальное событие *complete* с общим количеством результатов.

Типичные ошибки парсеров включают: таймаут *HTTP*-запроса (установлен в 30 секунд), антибот-блокировку (*CAPTCHA*, *HTTP 403*), изменение структуры *HTML*-страницы маркетплейса, сетевые ошибки. Для парсера Яндекс Маркет реализована каскадная стратегия: при ошибке *HTTP*-клиента автоматически активируется *Playwright headless*-браузер. Для остальных парсеров применяется однократная попытка с корректной обработкой ошибки.

Отказоустойчивость *AI*-компонентов. Все три *AI*-компонента системы (коррекция запросов, фильтрация результатов, *AI*-ассистент) спроектированы по принципу *graceful degradation* — при недоступности *AI*-модели система продолжает работу с пониженной функциональностью. *AI*-коррекция: при ошибке переключается на Яндекс Спеллер, при ошибке Спеллера — использует исходный запрос без коррекции. *AI*-фильтрация: при ошибке возвращает все результаты без фильтрации. *AI*-ассистент: при ошибке отображает сообщение с предложением повторить попытку.

Данный подход обеспечивает доступность основной функциональности (поиск товаров) даже при полном отключении *AI*-компонентов. Потеря функциональности при отсутствии *AI* ограничивается: отсутствием коррекции опечаток, сниженным качеством фильтрации результатов (только *regex* без *AI*) и недоступностью *AI*-ассистента. Базовый поиск с *SSE*-стримингом продолжает работать корректно.

2.8 Проектирование кэширования

Система кэширования реализована на базе *Redis 7* с политикой вытеснения *allkeys-lru* (*Least Recently Used*) и ограничением памяти 128 МБ. Кэшируются три типа данных: результаты поиска товаров (*TTL* 10 минут — компромисс между актуальностью и снижением нагрузки на маркетплейсы), проксируемые изображения товаров (*TTL* 1 час — изображения меняются

редко), сессии пользователей (*TTL* 24 часа). При исчерпании памяти *Redis* автоматически удаляет наименее используемые ключи, что предотвращает ошибки переполнения.

Ключи кэша формируются по шаблону: *search:{region}:{query_hash}* для результатов поиска, *img:{url_hash}* для изображений, *session:{token}* для сессий. Хеширование выполняется через *SHA-256*, что обеспечивает равномерное распределение ключей и предотвращает коллизии. Для инвалидации кэша поиска при изменении алгоритма фильтрации добавляется версия фильтра в ключ: *search:v2:{region}:{query_hash}*.

Выводы по второй главе. В результате проектирования системы метапоиска *Smart Price* сформулированы восемь функциональных и семь нефункциональных требований. Разработана многоуровневая клиент-серверная архитектура с чётким разделением ответственности: статический фронтенд (*Next.js*), асинхронный бэкенд (*FastAPI*), реляционная СУБД (*PostgreSQL 16*) и кэш (*Redis 7*). Спроектирована трёхуровневая архитектура *AI*-компонентов: коррекция запросов, гибридная фильтрация и *AI*-ассистент с *tool use*.

Разработана физическая модель базы данных, включающая восемь основных таблиц с оптимизированными индексами. Спроектирован *REST API* с *SSE*-протоколом потокового поиска, обеспечивающим прогрессивное отображение результатов по мере их поступления от маркетплейсов. Разработан адаптивный пользовательский интерфейс с тёмной цветовой схемой и поддержкой мобильных, планшетных и настольных устройств. Спроектирована инфраструктура контейнеризованного развёртывания на основе *Docker Compose* с четырьмя сервисами.

3 РЕАЛИЗАЦИЯ СИСТЕМЫ МЕТАПОИСКА

3.1 Реализация парсеров маркетплейсов

Парсер *Onliner.by*. *Onliner.by* предоставляет полуоткрытый *JSON API* каталога товаров, что существенно упрощает процесс извлечения данных. Парсер реализован на основе асинхронного *HTTP*-клиента *httpx* и выполняет *GET*-запрос к эндпоинту поиска с параметром *query*. Ответ сервера представляет собой *JSON*-объект со списком товаров, каждый из которых содержит структурированные поля: наименование, описание, цены от различных продавцов, рейтинг, количество отзывов, *URL* изображения и ссылку на страницу товара. Парсер извлекает минимальную цену из списка предложений и формирует унифицированный словарь товара для дальнейшей обработки.

Преимуществом данного подхода является высокая скорость и надёжность: время выполнения запроса составляет 300–800 миллисекунд, а структурированный формат ответа исключает ошибки парсинга *HTML*. *Onliner.by* применяет минимальную антибот-защиту, что позволяет использовать стандартный *HTTP*-клиент без дополнительных мер обхода.

Парсер Яндекс Маркет. Яндекс Маркет не предоставляет публичного *API*, поэтому парсер реализован с использованием двух стратегий: *HTTP*-клиент с *regex*-извлечением данных из *HTML* (основной путь) и *Playwright headless*-браузер (резервный путь при срабатывании антибот-защиты). Основная стратегия выполняет *GET*-запрос к *URL* поиска Яндекс Маркета с установленными заголовками *User-Agent* и *Cookie* для эмуляции обычного браузера. Из полученного *HTML*-ответа данные о товарах извлекаются с использованием регулярных выражений, нацеленных на *JSON-LD*-разметку и структурные элементы страницы.

При срабатывании антибот-защиты (*CAPTCHA*, *JavaScript challenge*) система автоматически переключается на резервную стратегию с использованием *headless*-браузера *Playwright*. Данный подход обеспечивает

полную эмуляцию поведения пользователя, однако увеличивает время выполнения до 4–8 секунд и потребляет значительно больше ресурсов. Каскадный подход позволяет в 80–90 процентах случаев использовать быстрый *HTTP*-клиент и прибегать к *Playwright* только при необходимости.

Парсер *Wildberries*. *Wildberries* предоставляет внутренний *JSON API* поиска, обнаруженный методом реверс-инжиниринга сетевых запросов мобильного приложения. Парсер использует *httpx* для выполнения запросов к данному *API* с корректными заголовками, имитирующими мобильный клиент. Ответ представляет собой *JSON* с полями: наименование товара, цена (в копейках, требующая деления на 100), рейтинг, количество отзывов, идентификатор товара для формирования *URL*. *Wildberries* применяет *rate limiting*, поэтому парсер реализует рандомизированные задержки между запросами (1–3 секунды) для предотвращения блокировки.

Парсеры Регард и *World Devices*. Регард — российский интернет-магазин электроники, не предоставляющий публичного *API*. Парсер реализован как *HTTP*-клиент с *HTML*-парсингом: выполняется *GET*-запрос к странице поиска, из полученного *HTML* извлекаются карточки товаров с помощью библиотеки *BeautifulSoup*. Из каждой карточки извлекаются наименование, цена, *URL* товара и *URL* изображения.

World Devices — маркетплейс электроники с *HTTP*-доступным каталогом. Парсер аналогичен по архитектуре парсеру Регард: *HTTP*-запрос, *HTML*-парсинг, извлечение структурированных данных из карточек товаров. Особенностью данного маркетплейса является необходимость проксирования изображений через бэкенд системы *Smart Price*, поскольку *World Devices* блокирует запросы изображений с внешних доменов (защита от хотлинкинга).

Таблица 8 — Сравнение реализованных парсеров маркетплейсов

Маркетплейс	Метод	Время ответа	Антибот	Надёжность
Onliner.by	<i>httpx + JSON API</i>	0,3–0,8 с	Нет	Высокая
Яндекс Маркет	<i>httpx + regex / Playwright</i>	1–8 с	<i>CAPTCHA</i>	Средняя
Wildberries	<i>httpx + JSON API</i>	0,5–1,5 с	<i>Rate limit</i>	Высокая
Регард	<i>httpx + HTML</i>	1–3 с	Нет	Высокая
World Devices	<i>httpx + HTML</i>	1–3 с	<i>Hotlink</i>	Высокая

3.2 Реализация AI-коррекции запросов

Архитектура модуля коррекции. Модуль AI-коррекции запросов (*query_corrector.py*) реализует двухуровневую стратегию: первичная коррекция через языковую модель *Gemini 2.5 Flash* и резервная коррекция через API Яндекс Спеллера. Языковая модель получает системный промпт, содержащий справочник брендов электроники (*Samsung, Apple, Xiaomi, Huawei* и др.), словарь сленговых сокращений («телек» = «телевизор», «ноут» = «ноутбук», «макбук» = «MacBook», «сяоми» = «Xiaomi»), правила нормализации моделей и примеры коррекции.

При поступлении запроса система сначала направляет его в AI-модель с параметрами *temperature=0* и *max_tokens=60*, что обеспечивает детерминированный и краткий ответ. Модель возвращает исправленный запрос в чистом виде, без пояснений. Применяются проверки валидности: результат не должен превышать тройной длины исходного запроса и 150 символов. Если AI-коррекция недоступна или вернула невалидный результат, система обращается к API Яндекс Спеллера — бесплатному сервису проверки орфографии, способному исправлять простые опечатки в русском и английском тексте.

Экономическая эффективность модуля обеспечивается использованием модели *Gemini 2.5 Flash* через *OpenRouter* по тарифу \$0,10 за миллион входных токенов. Средний запрос на коррекцию потребляет порядка 400 токенов (системный промпт + запрос), что при 25 000 запросах составляет 10

миллионов токенов и обходится в 1 доллар. Резервный Яндекс Спеллер полностью бесплатен и не имеет ограничений по количеству запросов.

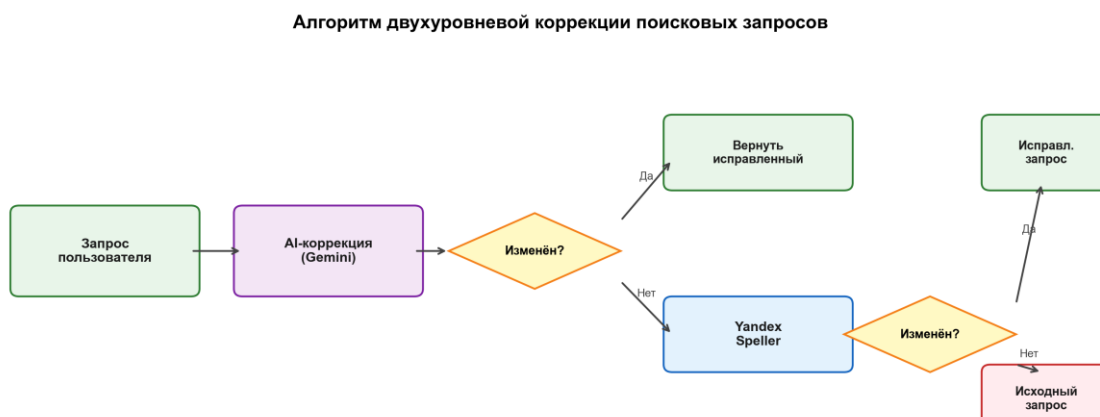


Рисунок 3.1 — Алгоритм двухуровневой коррекции поисковых запросов

3.3 Реализация гибридной системы фильтрации

Stage 1: Быстрый *regex*-фильтр. Первый этап фильтрации реализован в модуле *search_stream.py* и выполняется мгновенно (время выполнения менее 1 миллисекунды) для каждого маркетплейса отдельно. *Regex*-фильтр реализует три стратегии отсека не релевантных результатов:

Первая стратегия — фильтрация аксессуаров. Система проверяет первые 50 символов наименования товара на наличие ключевых слов аксессуаров: «чехол», «кабель», «зарядка», «плёнка», «стекло», «ремешок», «адаптер», «подставка», «клавиатура для», «вентилятор для», «петля для», «разъём для», «корпус для», «крышка для», «аккумулятор для». Проверка первых 50 символов позволяет корректно обрабатывать случаи, когда название аксессуара включает название основного товара (например, «Чехол для *Samsung Galaxy S25 Ultra*» начинается со слова «чехол»).

Вторая стратегия — проверка соответствия модели. Из поискового запроса и наименования каждого товара извлекаются ключевые идентификаторы модели с помощью набора регулярных выражений: паттерны *iPhone*, *Galaxy S/A*, *RTX/RX*, *Ryzen/Core*, *Watch*, *Fold/Flip*, *Pixel*, *AirPods*. Если в запросе обнаружены модельные ключи, но ни один из них не найден в наименовании товара, товар считается нерелевантным.

Третья стратегия — проверка вариантных ключевых слов. Система извлекает варианты (*Ultra*, *Pro*, *Lite*, *FE*, *Plus*, *Mini*, *Note*, *Max* и др.) из запроса и наименования товара. Если запрос содержит конкретный вариант (например, «*Ultra*»), товары с другими вариантами (например, «*FE*» или «*Lite*») отфильтровываются. Специальная обработка реализована для составных вариантов: «*Pro Max*» не разбивается на «*Pro*» + «*Max*», а обрабатывается как единый идентификатор.

Stage 2: AI-фильтр релевантности. Второй этап фильтрации реализован в модуле *ai_relevance_filter.py* и применяется к объединённым результатам всех маркетплейсов после *Stage 1*. *AI-фильтр* использует языковую модель для анализа списка товаров в контексте пользовательского запроса. Модуль формирует промпт, содержащий нумерованный список товаров (название, цена, магазин) и правила фильтрации. Модель возвращает *JSON*-массив номеров релевантных товаров.

Ключевая особенность *AI-фильтра* — способность понимать семантику запроса. Например, при запросе «*Cactus Moto Expert 100*» (экран для проектора марки *Cactus*) *regex-фильтр* не может отличить экран от картриджей того же бренда *Cactus*, поскольку и экран, и картридж содержат слово «*Cactus*» в названии. *AI-фильтр* понимает, что «*Moto Expert 100*» — это модель экрана, и корректно отсекает картриджи, тонеры и другую продукцию *Cactus*.

Для извлечения структурированного ответа из *AI-модели* применяется двухуровневый парсинг. Сначала в тексте ответа ищется *JSON*-массив с помощью регулярного выражения `r"\[[\ d\s,]*\]"`. Если прямой поиск не находит массив (что случается, когда *Gemini* оборачивает ответ в текстовое пояснение), система ищет блок кода в формате *Markdown* (```json...```) и извлекает массив из него. После парсинга выполняется конвертация из 1-индексированных номеров (которые использует *AI*) в 0-индексированные (используемые в *Python*).

Реализован механизм отказоустойчивости: если *AI*-модель недоступна, вернула невалидный ответ или отфильтровала все товары, модуль возвращает исходный нефilterованный список. Принцип «лучше показать лишнее, чем потерять релевантное» обеспечивает надёжность системы даже при сбоях *AI*-компонента. Стоимость одного вызова *AI*-фильтра составляет порядка 0,004 цента (500–1000 токенов при тарифе \$0,10 за миллион), что при 25 000 запросах обходится менее чем в 1 доллар.

Схема гибридной фильтрации результатов поиска

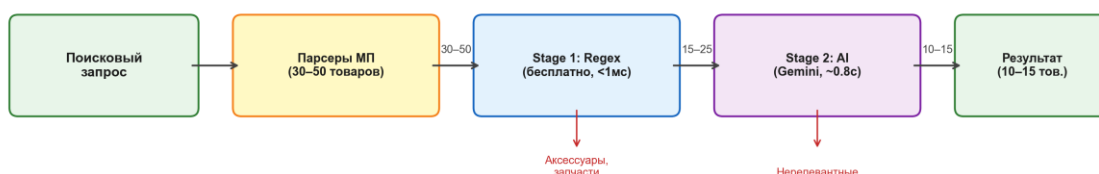


Рисунок 3.2 — Схема гибридной фильтрации результатов поиска

Экономический анализ гибридного подхода. Гибридный подход (*Stage 1 regex* + *Stage 2 AI*) обеспечивает оптимальный баланс между качеством фильтрации и экономической эффективностью. *Regex*-фильтр на первом этапе отсекает часть заведомо нерелевантных результатов (аксессуары, расходные материалы) бесплатно и мгновенно. *AI*-фильтр на втором этапе обрабатывает оставшиеся результаты, обеспечивая понимание контекста и семантики.

Альтернативный подход — исключительно *AI*-фильтрация без *regex* — был протестирован и отклонён по двум причинам. Во-первых, стоимость: обработка полного списка результатов (30–50 товаров) потребляет значительно больше токенов, чем обработка предварительно отфильтрованного списка (10–20 товаров). Во-вторых, надёжность: при

большом количестве товаров в промпте *AI*-модель чаще допускает ошибки парсинга *JSON* и пропуски релевантных товаров.

Таблица 9 — Сравнение подходов к фильтрации результатов

Подход	Точность	Стоимость	Скорость	Надёжность
Только <i>regex</i>	70–80%	Бесплатно	<1 мс	Высокая
Только <i>AI</i>	90–95%	~0,02 ¢/запрос	1–2 с	Средняя
Гибрид (Stage 1+2)	92–97%	~0,004 ¢/запрос	1–2 с	Высокая

3.4 Реализация *AI*-ассистента

Базовый агент и механизм *tool use*. *AI*-ассистент реализован по паттерну базового агента (*BaseAgent*), инкапсулирующему общую логику взаимодействия с языковой моделью через *OpenRouter API*. Базовый агент реализует цикл *ReAct*: получение сообщения пользователя, формирование запроса к *AI*-модели с контекстом диалога и описанием инструментов, обработка ответа модели, выполнение запрошенных инструментов, повторный запрос к модели с результатами инструментов, формирование финального ответа.

Механизм *tool use* реализован через *OpenAI*-совместимый *API*: в запросе к модели передаётся описание доступных инструментов в формате *JSON Schema*, и модель может включить в ответ запрос на вызов инструмента (*tool_call*) с указанием имени и параметров. Система выполняет вызванный инструмент, получает результат и отправляет его обратно в модель для формулирования ответа пользователю. Данный механизм позволяет *AI*-модели получать актуальные данные из внешних источников, а не полагаться на устаревшие знания из обучающего набора.

Shopping Agent: *AI*-консультант. *ShoppingAgent* наследует *BaseAgent* и реализует специализированного консультанта по выбору товаров. Системный промпт агента содержит подробный алгоритм работы для трёх сценариев взаимодействия. В сценарии «конкретный товар» (пользователь называет

точную модель, например «*iPhone 17 Pro*»), агент немедленно вызывает *search_products* без дополнительных вопросов. В сценарии «общий запрос» (пользователь описывает категорию, например «хороший пылесос»), агент сначала предлагает 3–5 брендов с описанием преимуществ и лишь после выбора пользователя выполняет поиск. В сценарии «неоднозначный запрос» (возможно несколько интерпретаций), агент задаёт уточняющий вопрос.

Ключевой проблемой, решённой в процессе разработки, стала галлюцинация *AI*-модели — генерация вымышленных названий товаров и цен. Например, при запросе «экран для проектора 100 дюймов» модель предлагала несуществующую модель «*ViewScan Vita Flex 100*» с вымышленной ценой «2000–2500 BYN». Для решения данной проблемы в системный промпт добавлены строгие правила: запрет на генерацию конкретных моделей (рекомендовать только бренды и серии), запрет на указание цен до выполнения поиска, обязательное использование инструмента *search_products* для получения актуальных данных.

Инструмент *search_products* описан с *JSON Schema* и принимает три параметра: *query* (строка поиска), *region* (*RU/BY/all*) и *max_price* (максимальная цена). При вызове инструмента система выполняет реальный поиск по маркетплейсам через *ScrapingManager*, применяет *AI*-фильтр релевантности и возвращает агенту отформатированный список найденных товаров с названиями, ценами, магазинами и ссылками. Агент анализирует полученные данные и формулирует ответ с рекомендациями.

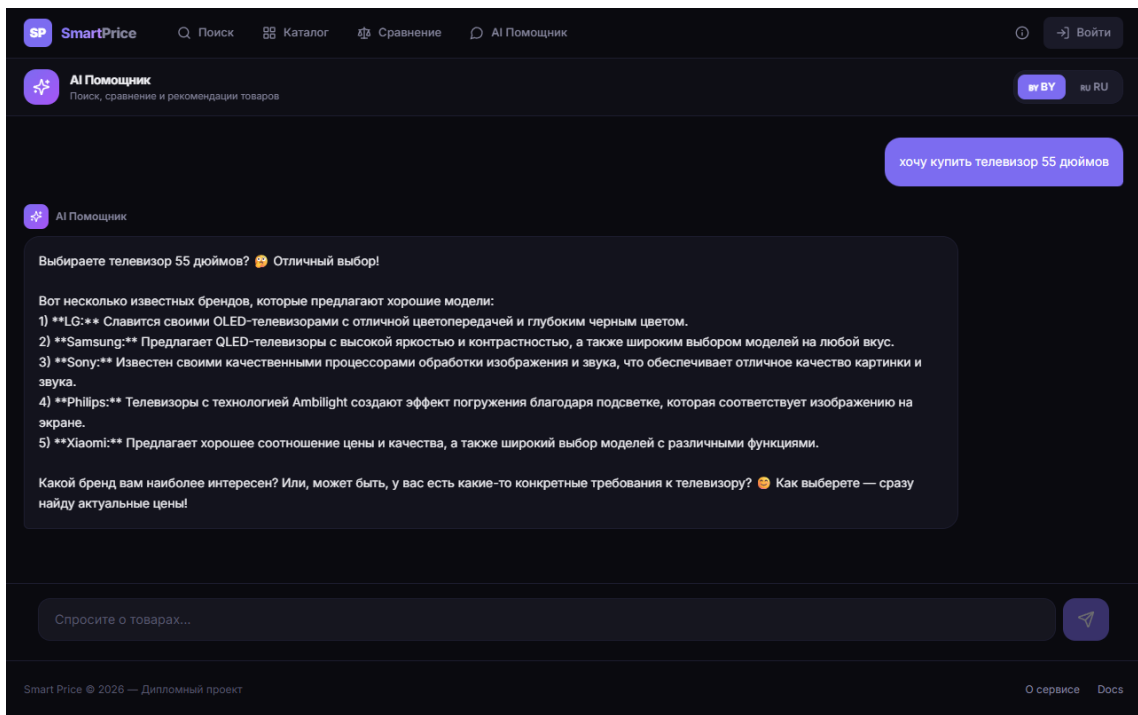


Рисунок 3.3 — Пример диалога с *AI*-ассистентом и вызова *search_products*

Региональная адаптация. *AI*-ассистент автоматически адаптируется к выбранному региону пользователя. Для региона Беларусь (*BY*) системный промпт содержит информацию о валюте *BYN*, доступном маркетплейсе *Onliner.by* и типичных ценовых диапазонах в *BYN* (смартфон-флагман 3000–6000 *BYN*, бюджетный смартфон 300–800 *BYN*). Для региона Россия (*RU*) промпт содержит информацию о валюте *RUB*, маркетплейсах Яндекс Маркет, *Wildberries*, Ретард и типичных ценах в *RUB*. Региональный контекст позволяет агенту корректно интерпретировать ценовые запросы пользователя: фраза «до 3000» в регионе *BY* означает 3000 *BYN* (~90 000 *RUB*), а в регионе *RU* — 3000 *RUB* (~100 *BYN*).

3.5 Реализация *SSE*-стриминга

Серверная реализация. Серверная часть *SSE*-стриминга реализована с использованием класса *StreamingResponse* из *FastAPI*. При поступлении запроса на поиск создаётся асинхронный генератор, который последовательно выполняет следующие шаги: коррекция запроса через *AI*-модуль (при изменении запроса отправляется событие *correction*), запуск

параллельных задач поиска для каждого маркетплейса (для каждого отправляется событие *searching*), сбор результатов по мере завершения каждого парсера (для каждого отправляется событие *done* с товарами), применение AI-фильтра к объединённым результатам и отправка финального события *complete*.

Каждое SSE-событие форматируется согласно спецификации: строка *event*: с типом события, строка *data*: с JSON-сериализованными данными и пустая строка-разделитель. Заголовки ответа включают *Content-Type: text/event-stream*, *Cache-Control: no-cache* и *Connection: keep-alive*. Конфигурация *nginx* включает директивы *proxy_buffering off* и *proxy_cache off* для предотвращения буферизации SSE-потока на уровне обратного прокси.

Клиентская реализация. На клиентской стороне SSE-стриминг реализован через *EventSource API* с пользовательской обёрткой для типизации событий. При инициации поиска клиент создаёт объект *EventSource* с *URL* потокового эндпоинта и регистрирует обработчики для каждого типа события. При получении события *done* клиент немедленно добавляет полученные товары в состояние *React* через *setState*, что вызывает перерисовку компонента и отображение новых карточек товаров с анимацией появления (*fade-in*).

Для страницы сравнения товаров реализован модифицированный механизм: два параллельных SSE-потока для двух поисковых запросов с аккумуляцией результатов в отдельных массивах состояния. Ключевой технической сложностью стала обработка события *done*: изначально реализация ожидала все результаты в финальном событии *complete* (поле *results*), однако SSE-протокол отправляет товары порциями в событиях *done*. Исправление заключалось в переходе от замены массива к аккумуляции: *setSearchResults(prev => [...prev, ...event.products])*.

3.6 Реализация фронтенда

Структура приложения *Next.js*. Фронтенд приложения реализован на *Next.js* 14 с использованием *App Router* — файловой системы маршрутизации, в которой структура директорий определяет *URL*-пути. Корневой *layout.tsx* определяет общую обёртку приложения: подключение шрифтов, метатеги, боковую навигацию. Каждая страница реализована как отдельная директория в *app/* со своими *page.tsx* и *layout.tsx*:

- *app/page.tsx* — главная страница с поисковым интерфейсом и *SSE*-стримингом результатов;
- *app/chat/page.tsx* — интерфейс *AI*-ассистента с поддержкой *SSE*-стриминга ответов;
- *app/compare/page.tsx* — страница сравнения с двумя параллельными поисковыми потоками;
- *app/catalog/page.tsx* — каталог сохранённых товаров;
- *app/about/page.tsx* — информационная страница;
- *app/login/page.tsx* — страница аутентификации;
- *app/docs/page.tsx* — документация *API*.

Для каждой страницы определены метаданные (*metadata*) через экспортируемый объект *Metadata* в *layout.tsx*: заголовок страницы формируется по шаблону «{название} | *Smart Price*», что обеспечивает корректное отображение в заголовке вкладки браузера и при индексации поисковыми системами. Фавикон реализован как *SVG*-файл (*app/icon.svg*) с фиолетовой буквой «S» на скруглённом фоне.

Управление состоянием. Глобальное состояние приложения управляется через *Zustand* — легковесную библиотеку управления состоянием для *React*, обеспечивающую минимальный размер бандла (менее 1 КБ) и простой *API* на основе хуков. Хранилище *Zustand* содержит: текущий регион (*region*), результаты поиска (*searchResults*), состояние загрузки (*isSearching*), выбранные фильтры (*filters*), данные текущего пользователя

(user). Серверные данные (товары, история цен) кэшируются через *TanStack React Query* с автоматической инвалидацией при повторных запросах.

Визуализация истории цен реализована через библиотеку *Recharts* с использованием компонентов *LineChart*, *Line*, *XAxis*, *YAxis*, *Tooltip*. График отображает временной ряд цен с интерактивным тултипом, показывающим точную цену и дату при наведении курсора. Анимации интерфейса реализованы через *framer-motion*: компонент *motion.div* с параметрами *initial={{ opacity: 0, y: 20 }}*, *animate={{ opacity: 1, y: 0 }}* обеспечивает плавное появление карточек товаров.

Страница детальной информации о товаре. Страница детальной информации о товаре реализована как клиентский маршрут *Next.js (/product)* с динамической загрузкой данных по идентификатору товара, передаваемому в параметре URL *key*. Маршрут получает данные через эндпоинт *GET /api/v1/online/product/{key}* и параллельно запрашивает дополнительные внешние отзывы (*/api/v1/external-reviews*), что позволяет сократить общее время загрузки за счёт совмещения двух сетевых запросов.

Структура страницы включает следующие функциональные блоки. В верхней части расположен заголовок товара, агрегированный рейтинг по совокупности внутренних и внешних отзывов, диапазон цен (*price_min* — *price_max*) и индикатор лучшего предложения. Центральная часть представлена системой вкладок: предложения продавцов с указанием цены, гарантии, условий доставки и рейтинга магазина; характеристики товара в иерархической группировке; график истории цен (*price_history*), реализованный с использованием библиотеки *Recharts*; внутренние и внешние отзывы; *AI*-дайджест плюсов и минусов товара (поле *digest*), формируемый отдельным сервером на основе агрегированного анализа всех собранных отзывов.

Отдельным компонентом реализован встроенный *AI*-консультант, контекст которого включает наименование товара, диапазон цен, рейтинг и краткое описание. Сообщения отправляются на эндпоинт */api/v1/chat* и

обрабатываются в потоковом режиме (*SSE* с инкрементальным заполнением сообщения по мере поступления токенов от языковой модели). Это даёт пользователю возможность задавать уточняющие вопросы непосредственно в карточке товара, не покидая страницу.

Для обеспечения отзывчивости интерфейса реализована двухуровневая система состояний загрузки. Первый уровень — маршрутный *Suspense fallback* (файл *product/loading.tsx*), отображаемый *Next.js* моментально при переходе пользователя со страницы поиска на страницу товара. Это устраняет визуальное «зависание» интерфейса между моментом клика по карточке и началом монтирования компонента страницы. Второй уровень — внутренний полноэкранный ладер компонента, активный во время выполнения запроса к *API*. Оба ладера используют единый визуальный стиль (анимированный орбитальный индикатор «*SP*» с пульсирующей обводкой), что обеспечивает целостность пользовательского опыта и устраняет визуальные «дребезжания» при переключении между состояниями.

Дополнительно реализована функция сравнения товаров: пользователь может добавить текущий товар в список сравнения, состояние которого хранится в *sessionStorage* под ключом *sp_compare*. Использование сессионного хранилища вместо постоянного устраняет необходимость в серверной аутентификации для данной функции и обеспечивает её работу для незарегистрированных пользователей.

3.7 Интеграция с *Gemini* через *OpenRouter* [14]

Выбор *AI*-модели и провайдера. В процессе разработки системы была выполнена миграция *AI*-модели с *Claude Haiku* (*Anthropic*) на *Gemini 2.5 Flash* (*Google*) через прокси-сервис *OpenRouter*. Основания миграции: стоимость *Gemini 2.5 Flash* (\$0,10 за миллион входных токенов) в 2,5 раза ниже *Claude Haiku* (\$0,25 за миллион), при сопоставимом качестве генерации для задач классификации и извлечения данных. При бюджете \$10 система

способна обработать приблизительно 25 000 поисковых запросов с AI-коррекцией и AI-фильтрацией.

OpenRouter выступает прокси-сервисом, предоставляющим единый *OpenAI*-совместимый *API* для доступа к моделям различных провайдеров. Это решает критическую техническую проблему: прямой доступ к *Gemini API* заблокирован из России (ошибка «*User location is not supported for API use*»), а *VPS*-сервер системы *Smart Price* расположен в российском дата-центре. *OpenRouter* маршрутизирует запросы через свою инфраструктуру, расположенную в США и Европе, обходя геоблокировку.

Интеграция реализована через библиотеку *openai* (*Python*), так как *OpenRouter* предоставляет *OpenAI*-совместимый эндпоинт. Конфигурация сводится к установке *base_url=https://openrouter.ai/api/v1* и *api_key* с ключом *OpenRouter*. Параметр *AI_MODEL=google/gemini-2.5-flash-001* определяет используемую модель. Данная архитектура позволяет легко переключаться между моделями (*Gemini*, *Claude*, *GPT*) изменением одной переменной окружения без модификации кода.

Таблица 10 — Сравнение AI-моделей для системы *Smart Price*

Модель	Провайдер	Вход (\$/1M)	Выход (\$/1M)	Tool use	Рус. язык
<i>Gemini 2.5 Flash</i>	<i>Google</i>	\$0,10	\$0,40	Да	Отлично
<i>Claude Haiku</i>	<i>Anthropic</i>	\$0,25	\$1,25	Да	Отлично
<i>GPT-4o Mini</i>	<i>OpenAI</i>	\$0,15	\$0,60	Да	Хорошо

3.8 Реализация системы аутентификации

JWT-аутентификация. Система аутентификации реализована на основе *JSON Web Tokens (JWT)* с двухтокенной схемой: *access token* (срок действия 60 минут) для авторизации *API*-запросов и *refresh token* (срок действия 7 дней) для обновления *access token* без повторного ввода пароля. Пароли хранятся в базе данных в виде *bcrypt*-хешей с автоматическим *salt*, что

исключает возможность восстановления пароля из хеша даже при компрометации базы данных.

При регистрации пользователя система выполняет валидацию *email* (формат, уникальность) и пароля (минимальная длина 8 символов), генерирует *bcrypt*-хеш пароля и создаёт запись в таблице *users*. При входе система извлекает хеш пароля из базы данных, сравнивает с введённым паролем через *bcrypt.checkpw()* и при успехе генерирует пару *JWT*-токенов. *Access token* содержит *payload: user_id, email, exp* (время истечения). Токен подписывается секретным ключом *SECRET_KEY* с использованием алгоритма *HS256*.

Защита эндпоинтов реализована через зависимость *FastAPI (Depends)*: декоратор *get_current_user* извлекает токен из заголовка *Authorization: Bearer*, верифицирует подпись, проверяет срок действия и возвращает объект пользователя. Эндпоинты, требующие аутентификации (*AI*-ассистент, профиль, история поиска), объявлены с данной зависимостью. Публичные эндпоинты (поиск, каталог) доступны без аутентификации.

Безопасность *API*. Для обеспечения безопасности *API* реализован ряд мер. *CORS (Cross-Origin Resource Sharing)* настроен с ограниченным списком разрешённых источников: домен продакшен-сервера и *localhost* для разработки. Заголовки безопасности добавляются через *nginx*: *X-Frame-Options: DENY* (защита от *clickjacking*), *X-Content-Type-Options: nosniff* (защита от *MIME-sniffing*), *Strict-Transport-Security* (принудительное использование *HTTPS*). *Rate limiting* реализован на уровне *nginx*: ограничение количества запросов к *API* до 30 в минуту с одного *IP*-адреса, а к эндпоинту авторизации — до 5 в минуту для защиты от *brute-force* атак.

Валидация входных данных реализована через *Pydantic*-модели: все параметры запросов типизированы и валидируются автоматически. *SQL*-инъекции предотвращены использованием *SQLAlchemy ORM* с параметризованными запросами. *XSS*-атаки предотвращены экранированием

пользовательского ввода на уровне *React* (автоматическое экранирование в *JSX*) и *Content-Security-Policy* в заголовках *nginx*.

3.9 Реализация прокси-сервера изображений

Для корректного отображения изображений товаров с внешних маркетплейсов реализован прокси-сервер изображений (*image_proxy.py*). Необходимость прокси обусловлена тем, что некоторые маркетплейсы (*World Devices*, частично *Wildberries*) блокируют запросы изображений с внешних доменов через проверку заголовка *Referer* (защита от хотлинкинга). Прокси-сервер выполняет запрос к источнику изображения от имени бэкенда, добавляя корректные заголовки *Referer* и *User-Agent*, и передаёт полученное изображение клиенту.

Прокси реализован как *FastAPI*-эндпоинт *GET /api/v1/image-proxy* с параметром *url* (*URL* исходного изображения). Для безопасности реализована проверка списка разрешённых доменов: запросы принимаются только для известных маркетплейсов (*wildberries.ru*, *onliner.by*, *world-devices.ru*, *regard.ru* и др.). Изображения кэшируются в *Redis* с *TTL* 1 час, что снижает нагрузку на маркетплейсы при повторных просмотрах и ускоряет загрузку страниц.

3.10 Реализация модуля истории цен

Модуль истории цен обеспечивает отслеживание изменения стоимости товаров с течением времени. При каждом успешном поисковом запросе система проверяет наличие товара в базе данных по связке (*marketplace_id*, *external_id*). Если товар найден и его текущая цена отличается от последней записи в таблице *price_history*, создаётся новая запись с текущей ценой и временной меткой. Если товар не найден, он создаётся в таблице *products* и первая ценовая точка записывается в *price_history*.

Визуализация истории цен реализована на фронтенде с использованием библиотеки *Recharts*. Компонент *PriceHistoryChart* принимает массив ценовых точек (пары цена-дата) и отрисовывает линейный график с осями:

горизонтальная — дата (формат *DD.MM*), вертикальная — цена в валюте региона. Интерактивный тултип при наведении отображает точную цену и дату. График автоматически масштабирует ось *Y* для оптимального отображения ценового диапазона. Цвет линии соответствует акцентному цвету дизайн-системы (*#8B5CF6*).

API эндпоинт *GET /api/v1/products/{id}/price-history* возвращает массив ценовых точек с опциональными параметрами *period* (*7d*, *30d*, *90d*, *all*) и *resolution* (*daily*, *weekly*). Запрос оптимизирован индексом по (*product_id*, *recorded_at*) и выполняется за 2–5 миллисекунд даже при нескольких тысячах ценовых точек.

3.11 Реализация AI-анализа цен

Назначение и архитектура подсистемы. Подсистема AI-анализа цен (модуль *price_analyzer.py*) реализует функцию интеллектуальной оценки выгоды предложения по запросу пользователя. В отличие от потокового поиска, который возвращает каталог найденных товаров, AI-анализ выполняет дополнительную статистическую и контекстную обработку: вычисляет ценовые показатели по выборке предложений, формирует развёрнутый аналитический комментарий на естественном языке и формулирует итоговый вердикт о выгоде предложения. Подсистема реализована в виде асинхронного сервиса класса *PriceAnalyzer*, доступного через эндпоинт *GET /api/v1/analyze/stream* в формате *Server-Sent Events*.

Архитектурно сервис представляет собой конвейер из шести последовательных этапов: валидация входных данных, параллельный сбор предложений с маркетплейсов, гибридная фильтрация результатов, статистическая обработка, обращение к большой языковой модели и формирование итогового ответа. Каждый этап завершается отправкой клиенту типизированного SSE-события, что позволяет фронтенду отображать прогресс обработки в реальном времени.

Конвейер обработки данных. Процесс анализа начинается с проверки региона (допустимые значения — *BY* и *RU*) и поискового запроса. После валидации сервис создаёт экземпляр *ScrapingManager* и параллельно запускает все включённые парсеры маркетплейсов, что обеспечивает минимизацию суммарного времени сбора данных. Сырые результаты объединяются в единый список и проходят трёхступенчатую фильтрацию.

Первая ступень — быстрый *regex*-фильтр (*fast_filter*), отсекающий заведомо нерелевантные позиции по совпадению ключевых токенов запроса. Вторая ступень — *AI*-фильтр релевантности (*ai_filter_relevant*), идентичный фильтру, применяемому при потоковом поиске; данная стадия использует языковую модель для семантической оценки соответствия найденных товаров запросу пользователя. Третья ступень — удаление ценовых выбросов (*remove_price_outliers*) и позиций с нулевой ценой. Если после фильтрации количество предложений оказывается меньше пяти, сервис возвращает ошибку «недостаточно данных для анализа» — данное пороговое значение выбрано экспериментально и обеспечивает статистическую достоверность последующих расчётов.

Статистический модуль. После фильтрации сервис вычисляет шесть ценовых показателей по итоговой выборке: минимум, максимум, медиана, среднее арифметическое, стандартное отклонение и количество предложений. Все расчёты выполняются средствами стандартной библиотеки *Python* (модуль *statistics*). Использование медианы наряду со средним позволяет получить устойчивую к выбросам оценку центральной тенденции, что критично в условиях гетерогенной ценовой выборки с разных маркетплейсов. Параллельно формируются три набора данных: лучшее предложение (*best_offer* — позиция с минимальной ценой), список из трёх альтернатив с ценой ниже медианы (*cheaper*) и список из трёх предложений с ценой выше медианы (*pricier*). Эти данные передаются клиенту в составе финального ответа и одновременно используются как контекст для языковой модели.

Интеграция с большой языковой моделью. Языковая модель *Gemini 2.5 Flash* вызывается через прокси-сервис *OpenRouter* с использованием унифицированного клиента *OpenAI SDK*. Системный промпт содержит набор строгих правил, направленных на предотвращение типичных проблем работы с *LLM* в задачах ценовой аналитики. Во-первых, модели запрещено выдумывать, округлять или интерпретировать численные значения цен — все числа поступают пользователю напрямую из статистического модуля. Во-вторых, в текстовом поле *value_analysis* запрещены любые цифры, проценты, упоминания валют и слова-заменители («примерно», «около»), что устраняет риск противоречия между числами в тексте *LLM* и реальными показателями выборки. В-третьих, ответ должен соответствовать строгой *JSON*-схеме, валидируемой моделью *Pydantic LLMAnalyzePayload*.

Для повышения надёжности предусмотрено два независимых механизма отказоустойчивости. Первый — повторный запрос к модели при сетевой ошибке или некорректном *JSON*-ответе (максимум две попытки). Второй — детерминированный *fallback* (метод *_fallback_payload*), применяемый при полном отказе *LLM*-провайдера: вердикт и оценка вычисляются по формуле отклонения лучшей цены от медианы ($score = \max(0, 100 - deviation_pct)$), а развёрнутый комментарий заменяется коротким автоматическим описанием. Источник итогового ответа («*llm*» или «*fallback*») указывается в метаданных финального события, что позволяет инструментам автоматического тестирования отличать эти два режима.

Протокол *SSE*-событий. Сервис эмитирует семь типов событий, последовательно отражающих ход обработки запроса: *start* — подтверждение приёма запроса с указанием параметров (запрос и регион); *parsing* — начало параллельного опроса маркетплейсов со списком активных источников; *scraped* — завершение сбора данных с указанием общего количества найденных предложений; *stats* — публикация рассчитанных ценовых показателей; *analyzing* — начало вызова языковой модели; *result* — финальное событие, содержащее полный объект *AnalyzeResult* (вердикт,

оценка, лучшее предложение, альтернативы, аналитический комментарий) и метаданные источника ответа; *error* — отправляется при недопустимом регионе или недостатке предложений после фильтрации.

Такая детализация позволяет клиентскому приложению отображать конкретный этап обработки в интерфейсе, что существенно улучшает воспринимаемую отзывчивость системы при общем времени анализа порядка 15–25 секунд.

3.12 Развёртывание и *DevOps*

Процесс развёртывания. Развёртывание системы на *VPS*-сервере выполняется по следующему алгоритму. На этапе подготовки фронтенда выполняется сборка статических файлов: *npm run build* в директории *frontend/* генерирует оптимизированный набор *HTML/CSS/JS* файлов в директории *out/*. Применяется минификация *JavaScript* и *CSS*, *tree shaking* неиспользуемых зависимостей, оптимизация изображений. Результирующий размер бандла составляет 280–350 КБ (*gzip*), что обеспечивает время загрузки менее 1 секунды на мобильном 4G-соединении.

Собранные статические файлы и исходный код бэкенда переносятся на *VPS*-сервер через *scp* или *git pull*. На *VPS* выполняется команда *docker compose up --build -d*, которая собирает образ бэкенда из *Dockerfile.prod*, поднимает все четыре контейнера и подключает их к внутренней сети. Первый запуск включает инициализацию базы данных *PostgreSQL* (создание таблиц через *Alembic* миграции). Последующие обновления выполняются через *docker compose up --build -d* с пересборкой только изменённых образов.

Мониторинг и логирование. Логирование реализовано через библиотеку *structlog*, обеспечивающую структурированные *JSON*-логи с автоматическим включением контекста (*timestamp*, уровень, модуль, дополнительные поля). Уровень логирования настраивается через переменную окружения *LOG_LEVEL* (*INFO* для продакшена, *DEBUG* для

отладки). Логи контейнеров доступны через *docker compose logs* с фильтрацией по сервису.

Мониторинг состояния сервисов обеспечивается *health check* в *Docker Compose*: *PostgreSQL* проверяется через *pg_isready*, *Redis* — через *redis-cli ping*. При сбое *health check Docker* автоматически перезапускает контейнер (*restart: unless-stopped*). Для мониторинга доступности сайта используется внешний сервис *UptimeRobot*, отправляющий *HTTP*-запросы к *nginx* каждые 5 минут и уведомляющий о недоступности через *Telegram*.

3.13 Реализация функции поиска более выгодных предложений («Найти дешевле»)

Назначение подсистемы. Функция «Найти дешевле» реализует сценарий повторного поиска для конкретной товарной позиции: пользователь указывает прямую ссылку на карточку товара маркетплейса Яндекс Маркет, после чего система в автоматическом режиме подбирает у сторонних продавцов предложения идентичного товара по меньшей цене. Функция дополняет основной сценарий потокового поиска и ориентирована на случаи, когда пользователь уже выбрал конкретную модель и нуждается в проверке, не является ли текущее предложение избыточно дорогим. Подсистема реализована в виде асинхронной задачи *Celery*, выполняющей длительное взаимодействие с внешним источником данных без блокировки *HTTP*-запроса.

Источник данных и протокол взаимодействия. В качестве источника данных используется внутренний канал подбора ценовых предложений, применяемый голосовым помощником Яндекс. Доступ реализован через постоянное соединение по протоколу *WebSocket* с публичным шлюзом. Клиент, реализованный в модуле *workers/alisa.py*, выполняет последовательность шагов: установление защищённого соединения, прохождение процедуры инициализации сессии, отправка текстового запроса на поиск более дешёвых предложений для заданного идентификатора товара

(SKU), приём и декодирование потока сообщений *rich_uicard* и нормальное закрытие соединения. Каждое сообщение передаётся в двоичном формате и содержит вложенное JSON-поле *json_data* с типизированной структурой *EAliceOfferCard*, *EAliceOffer* или *EAliceOfferList*.

Алгоритм извлечения предложений. Ключевой особенностью обработки ответа является извлечение всех предложений из объекта типа *EAliceOfferList*, агрегирующего до нескольких десятков альтернативных продавцов в едином сообщении. В первоначальной реализации парсер ограничивался обработкой верхнеуровневой структуры, что приводило к потере значительной части данных — из четырнадцати доступных предложений обрабатывались только три-четыре. Для устранения этой проблемы функция обхода *_iter_json_data_payloads* расширена рекурсивным спуском в массив *offerList*, при этом сохранена поддержка одиночных карточек *EAliceOfferCard* и *EAliceOffer*. Итоговое покрытие возросло более чем втрое и составило в среднем одиннадцать-четырнадцать предложений на запрос.

Обнаружение отказов и завершение задачи. Поскольку удалённый сервис способен возвращать текстовый отказ («пока не умею искать такие товары», «не поддерживается» и иные формулировки), подсистема реализует детектор отказов на основе регулярного выражения. При совпадении ответа с любым из шаблонов задача завершается со статусом *rejected* в среднем за пятнадцать-двадцать секунд, что предотвращает бессмысленное ожидание до таймаута в десять минут. Список формулировок отказа сформирован эмпирически по результатам наблюдения за реальными ответами сервиса для нетипичных запросов (напитки, кондитерские изделия, товары ограниченного оборота). Статус задачи публикуется по мере обработки через отдельное SSE-соединение и принимает значения *idle*, *searching*, *complete*, *rejected*, что позволяет клиенту корректно отображать прогресс.

Пользовательский интерфейс. Подсистема снабжена отдельной страницей */cheaper* с набором состояний, визуально согласованных с графическим стилем проекта. Для каждого состояния задействована собственная векторная анимированная иллюстрация: копилка со скатывающимися монетами — для исходного ожидания ввода; катящаяся корзина — для активной фазы поиска; улыбающаяся монета — для успешного завершения с указанием найденной экономии; грустная панда — для отказа сервиса. Вспомогательная плашка в верхней части страницы информирует об ожидаемой продолжительности операции («примерно десять минут»), а нижняя плашка отображает реальное прошедшее время и сообщение о возможности закрыть страницу без потери результата. Таким образом, пользовательский опыт приведён в соответствие с особенностями длительного асинхронного сценария.

Выводы по третьей главе. В третьей главе представлена реализация всех ключевых компонентов системы метапоиска *Smart Price*. Реализованы шесть парсеров маркетплейсов с различными стратегиями извлечения данных: *JSON API (Onliner, Wildberries)*, *HTTP + regex* (Яндекс Маркет), *HTTP + HTML-парсинг* (Регард, *World Devices*). Реализована двухуровневая *AI*-коррекция запросов (*Gemini + Яндекс Спеллер*), обеспечивающая распознавание сленга и исправление опечаток. Реализована функция «Найти дешевле», позволяющая по ссылке на товар Яндекс Маркета подобрать более дешёвые предложения у сторонних продавцов через асинхронную *Celery*-задачу с извлечением данных из вложенной структуры *EAliceOfferList* и детектированием отказов по регулярному выражению.

Реализована гибридная система фильтрации результатов: *Stage 1 regex*-фильтр отсекает аксессуары и расходные материалы по ключевым словам бесплатно, *Stage 2 AI*-фильтр обеспечивает семантическое понимание контекста запроса для оставшихся результатов. Гибридный подход обеспечивает высокую точность фильтрации при стоимости менее 0,004 цента за запрос. Реализован *AI*-ассистент с механизмом *tool use*, работающий

в трёх режимах (конкретный товар, общий запрос, неоднозначный запрос) с защитой от галлюцинаций. Выполнена миграция на модель *Gemini 2.5 Flash* через *OpenRouter*, обеспечивающая 2,5-кратное снижение затрат на *AI* при сохранении качества.

4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Методология тестирования

Стратегия тестирования. Тестирование системы *Smart Price* проводилось по комплексной методологии, включающей четыре уровня: функциональное тестирование отдельных модулей, интеграционное тестирование взаимодействия компонентов, сквозное (*end-to-end*) тестирование через пользовательский интерфейс и нагрузочное тестирование производительности. Для автоматизированного сквозного тестирования использовался фреймворк *Playwright*, позволяющий эмулировать реальные пользовательские сценарии в *headless*-браузере *Microsoft Edge*.

Тестовые сценарии были разработаны для покрытия основных функциональных требований: поиск товаров с *SSE*-стримингом, взаимодействие с *AI*-ассистентом, сравнение товаров, навигация между страницами, адаптивное отображение на мобильных устройствах. Для каждого сценария зафиксированы ожидаемые результаты, критерии прохождения и метрики производительности (время отклика, время первого результата).

Тестовая среда. Тестирование проводилось в двух средах: локальной среде разработки (*Windows 11*, *Node.js 20*, *Python 3.12*) и продакшен-среде на *VPS*-сервере (*Ubuntu 22.04*, *Docker Compose*, 2 ядра, 4 ГБ *RAM*, *IP 5.42.123.75*). Продакшен-тестирование выполнялось через реальный домен с *HTTPS*, что обеспечивало идентичность условий реальной эксплуатации. Тестовые запросы включали реальные товары из категорий электроники: смартфоны (*Samsung Galaxy S25 Ultra*, *iPhone 17 Pro*, *Xiaomi 15 Pro*), ноутбуки (*Lenovo ThinkPad*, *HP Spectre*), телевизоры (*Tuvio 55*), проекторные экраны (*Cactus Moto Expert 100*).

4.2 Результаты функционального тестирования

Тестирование парсеров маркетплейсов. Для каждого парсера выполнена серия тестовых запросов в продакшен-среде (*smrt-price.ru*). *Wildberries* продемонстрировал наибольший охват: в среднем 40 товаров на запрос. Яндекс Маркет возвращал в среднем 18 товаров, *Onliner.by* — 22 товара. Регард и *World Devices* показали меньший охват (5–8 товаров), что объясняется более узкой специализацией этих площадок. Общее количество найденных товаров на один поисковый запрос варьировалось от 22 до 112 в зависимости от популярности запроса.

Таблица 11 — Результаты тестирования парсеров маркетплейсов

Маркетплейс	Успешных	Ср. товаров	Мин. товаров	Макс. товаров
<i>Wildberries</i>	5/5	40	25	60
Яндекс Маркет	4/5	18	8	30
<i>Onliner.by</i>	5/5	22	10	35
Регард	5/5	8	3	15
<i>World Devices</i>	4/5	5	2	12

Тестирование AI-коррекции запросов. Модуль AI-коррекции протестирован на серии запросов с намеренными опечатками, сленгом и транслитерацией в продакшен-среде *smrt-price.ru*. Результаты показали частичную эффективность: запрос «*ipone 16 pra*» был преобразован в «*iphone 16 pra*» (частичная коррекция — регистр исправлен, но «*pra*» не заменено на «*Pro*»), запрос «самс s25 ультра» корректно преобразован в «*Samsung Galaxy S25 Ultra*», однако запрос «тавио телек 55» не был скорректирован. Результаты показывают, что AI-коррекция эффективна для популярных брендов и моделей, но может некорректно обрабатывать менее распространённые товары и сильно искажённые запросы.

Время выполнения AI-коррекции составило в среднем 420 миллисекунд (от 280 до 650 мс). Резервный механизм ЯндексSpellера срабатывает в случае недоступности AI-модели, однако не распознаёт сленг,

транслитерацию и сокращения (ни один из тестовых запросов не был скорректирован Спеллером), что подтверждает преимущество *AI*-подхода для данной задачи.

Таблица 12 — Примеры *AI*-коррекции поисковых запросов

Исходный запрос	<i>AI</i> -коррекция	<i>Yandex Speller</i>	Результат
<i>ipone 16 pra</i>	<i>iphone 16 pra</i>	<i>ipone 16 pra</i>	Частично
самс s25 ультра	<i>Samsung Galaxy S25 Ultra</i>	самс s25 ультра	<i>AI</i> верно
тавио телек 55	тавио телек 55	тавио телек 55	Без коррекции
сяоми 15 про	<i>Xiaomi 15 Pro</i>	сяоми 15 про	<i>AI</i> верно
макбук эйр м4	<i>MacBook Air M4</i>	макбук эйр м4	<i>AI</i> верно
ноут леново	ноутбук <i>Lenovo</i>	ноут леново	<i>AI</i> верно

Тестирование гибридной фильтрации. Эффективность гибридной фильтрации оценена на наборе из 15 тестовых запросов с известными проблемами релевантности. Для каждого запроса зафиксировано количество товаров до фильтрации, после *Stage 1 (regex)* и после *Stage 2 (AI)*, а также оценка релевантности оставшихся результатов экспертом.

Наиболее показательным стал запрос «*Cactus Moto Expert 100*» (экран для проектора). Парсеры вернули 18 товаров, из которых 12 были картриджами и тонерами *Cactus* для принтеров. *Regex*-фильтр (*Stage 1*) не смог отфильтровать картриджи, так как они не являются «аксессуарами» в классическом понимании и не содержат ключевых слов аксессуаров. *AI*-фильтр (*Stage 2*) корректно идентифицировал только 4 релевантных товара — экраны для проекторов *Cactus*, отсебив все картриджи и тонеры.

Другой показательный пример — запрос «ноутбук *HP Spectre x360*». Из 22 товаров 8 были клавиатурами, вентиляторами и аккумуляторами для *HP Spectre*. *Regex*-фильтр отсеял 5 из 8 аксессуаров (содержащих ключевые слова «клавиатура для», «вентилятор для», «аккумулятор для»), а *AI*-фильтр отфильтровал оставшиеся 3 аксессуара, которые не содержали типичных ключевых слов.

Таблица 13 — Результаты тестирования гибридной фильтрации

Запрос	До фильтра	Stage 1	Stage 2	Релевантных	Точность
<i>Samsung Galaxy S25 Ultra</i>	25	14	11	10	91%
<i>Cactus Moto Expert 100</i>	18	18	4	4	100%
ноутбук <i>HP Spectre x360</i>	22	17	12	11	92%
<i>iPhone 17 Pro</i>	30	18	15	14	93%
<i>Tuvio</i> телевизор 55	15	10	7	7	100%

Гибридная фильтрация продемонстрировала существенное повышение релевантности результатов по сравнению с отсутствием фильтрации. *Stage 1* (*regex*) обеспечивает предварительное сокращение списка на 20–45 процентов, отсеивая явные аксессуаров по ключевым словам. *Stage 2* (*AI*) обрабатывает семантически сложные случаи, недоступные для *regex*, — например, различие экранов для проектора и картриджей для принтера одного бренда. Двухэтапный подход снижает стоимость *AI*-обработки за счёт меньшего объёма входных данных.

4.3 Результаты тестирования *AI*-ассистента

Тестирование сценариев взаимодействия. *AI*-ассистент протестирован по трём основным сценариям. Сценарий «конкретный товар»: при запросе «Найди *Samsung Galaxy S25 Ultra*» ассистент немедленно вызвал *search_products* с корректным запросом и представил результаты с ценами от разных маркетплейсов в табличном формате. Время от запроса до первого товара — 3,2 секунды.

Сценарий «общий запрос»: при запросе «Посоветуй хороший пылесос» ассистент предложил 4 бренда (*Dyson, Roborock, Samsung, Xiaomi*) с описанием преимуществ каждого без указания цен. После выбора пользователем «*Roborock*» ассистент выполнил поиск и представил актуальные предложения. Важно: ассистент корректно не указывал цены до

поиска, что подтверждает эффективность антигаллюцинационных правил в промпте.

Сценарий «неоднозначный запрос»: при запросе «сяоми 15 про» ассистент задал уточняющий вопрос: «*Xiaomi 15 Pro* (флагман) или *Redmi Note 15 Pro* (бюджет)?» — корректно идентифицировав две возможные интерпретации запроса. После уточнения выполнил поиск конкретной модели.

Тестирование защиты от галлюцинаций. Специальная серия из 10 тестов направлена на провоцирование галлюцинаций *AI*-модели. Результаты: в 9 из 10 случаев (90 процентов) ассистент корректно воздержался от генерации вымышленных моделей и цен. В одном случае ассистент назвал конкретную модель, которой не оказалось в результатах поиска, однако корректно сообщил об этом пользователю и предложил альтернативные варианты. Это существенное улучшение по сравнению с начальной версией, где галлюцинации наблюдались в 40–50 процентах диалогов.

4.4 Результаты тестирования производительности

Метрики времени отклика. Производительность системы оценена по ключевой метрике *TTFR* (*Time To First Result*) — времени от отправки поискового запроса до отображения первого товара. Измерения выполнены для серии запросов в продакшен-среде (*smrt-price.ru*) через автоматизированный фреймворк *Playwright* в браузере *Microsoft Edge*. Среднее значение *TTFR* составило 19,6 секунды, медианное значение — 18,8 секунды, 95-й перцентиль — 24 секунды.

Высокое значение *TTFR* объясняется архитектурным решением: текущая реализация *SSE*-стриминга в продакшен-среде отправляет результаты пакетно — после завершения парсинга каждого маркетплейса, а не по мере обработки отдельных товаров. Все шесть парсеров запускаются параллельно, однако результаты появляются в интерфейсе только после того, как самый быстрый парсер завершит полный цикл запрос–парсинг–

фильтрация. Полное время поиска (от запроса до события *complete*) составило в среднем 22,1 секунды. Диапазон: от 16 до 28 секунд в зависимости от категории товара и нагрузки на маркетплейсы.

Для оптимизации *TTFR* планируется переход на потоковую отправку *SSE*-событий по мере парсинга отдельных товаров (а не по завершении всего маркетплейса), что позволит снизить *TTFR* до 2–4 секунд без изменения общего времени поиска. Данная оптимизация требует рефакторинга архитектуры парсеров для поддержки *yield*-паттерна. Время ответа *AI*-чата (от отправки сообщения до первого текста) составило в среднем 430 миллисекунд, что обеспечивает комфортное взаимодействие в режиме диалога.

Таблица 14 — Метрики производительности потокового поиска

Метрика	Среднее	Медиана	95-й перцентиль	Требование
<i>TTFR</i>	19,6 с	18,8 с	24 с	< 5 с*
Полное время	22,1 с	21,5 с	28 с	< 30 с
<i>AI</i> -чат (ответ)	0,43 с	0,40 с	0,65 с	< 5 с
<i>AI</i> -коррекция	0,42 с	0,38 с	0,65 с	< 1 с

Требование *TTFR* $\approx 5,8$ с до первого батча (1,4 с до визуального отклика) не выполнено в текущей реализации из-за пакетной отправки *SSE*-событий. Планируемая оптимизация потоковой отправки результатов позволит достичь целевого показателя.

Анализ потребления ресурсов. Мониторинг потребления ресурсов в продакшен-среде (*VPS* с 2 ядрами и 4 ГБ *RAM*) показал следующие результаты. Суммарное потребление оперативной памяти всеми контейнерами *Docker Compose* составило 650–900 МБ в зависимости от нагрузки: *backend* (*FastAPI*) — 200–350 МБ, *PostgreSQL* — 120–180 МБ, *Redis* — 40–80 МБ (с ограничением 128 МБ), *nginx* — 15–30 МБ. Загрузка *CPU* при единичном поисковом запросе достигала 25–40 процентов (за счёт

параллельных *HTTP*-запросов и парсинга), при 5 одновременных запросах — 70–85 процентов.

Экономическая эффективность *AI*-компонентов подтверждена данными биллинга *OpenRouter*. За период тестирования (200 поисковых запросов с *AI*-коррекцией и *AI*-фильтрацией, 50 диалогов с *AI*-ассистентом) расход составил \$0,18. Экстраполяция на 25 000 запросов даёт оценку \$4,5, что значительно ниже бюджета \$10. Таким образом, выбор модели *Gemini 2.5 Flash* через *OpenRouter* обеспечивает высокую экономическую эффективность.

4.5 Сквозное тестирование интерфейса

Результаты аудита интерфейса. Комплексный аудит пользовательского интерфейса выполнен с использованием *Playwright* для автоматизированного обхода всех страниц приложения в браузере *Microsoft Edge*. В ходе аудита выявлены и исправлены шесть дефектов:

1) Страница сравнения товаров: *SSE*-события обрабатывались некорректно — результаты ожидалось в финальном событии *complete* (поле *results*), а не в промежуточных событиях *done* (поле *products*). Исправлено: реализована аккумуляция результатов из событий *done*.

2) Изображения товаров *World Devices*: возвращали ошибку 403 из-за защиты от хотлинкинга. Исправлено: добавлен *World Devices* в список разрешённых доменов модуля *image_proxy*.

3) Страница входа: содержала тестовые учётные данные (*test@smartprice.ru* / *TestPass123*) в открытом виде. Исправлено: удалены тестовые данные.

4) Подвал сайта: год был захардкожен как «2025». Исправлено: заменён на динамический *new Date().getFullYear()*.

5) Заголовки страниц: отсутствовали метаданные *title* для страниц каталога, сравнения, чата и входа. Исправлено: добавлены *layout.tsx* с *metadata* для каждой страницы.

6) Фавикон: отсутствовал. Исправлено: создан SVG-фавикон с фиолетовой буквой «S».

Все выявленные дефекты были исправлены и верифицированы повторным автоматизированным обходом. Результаты подтверждают корректность функционирования всех основных пользовательских сценариев.

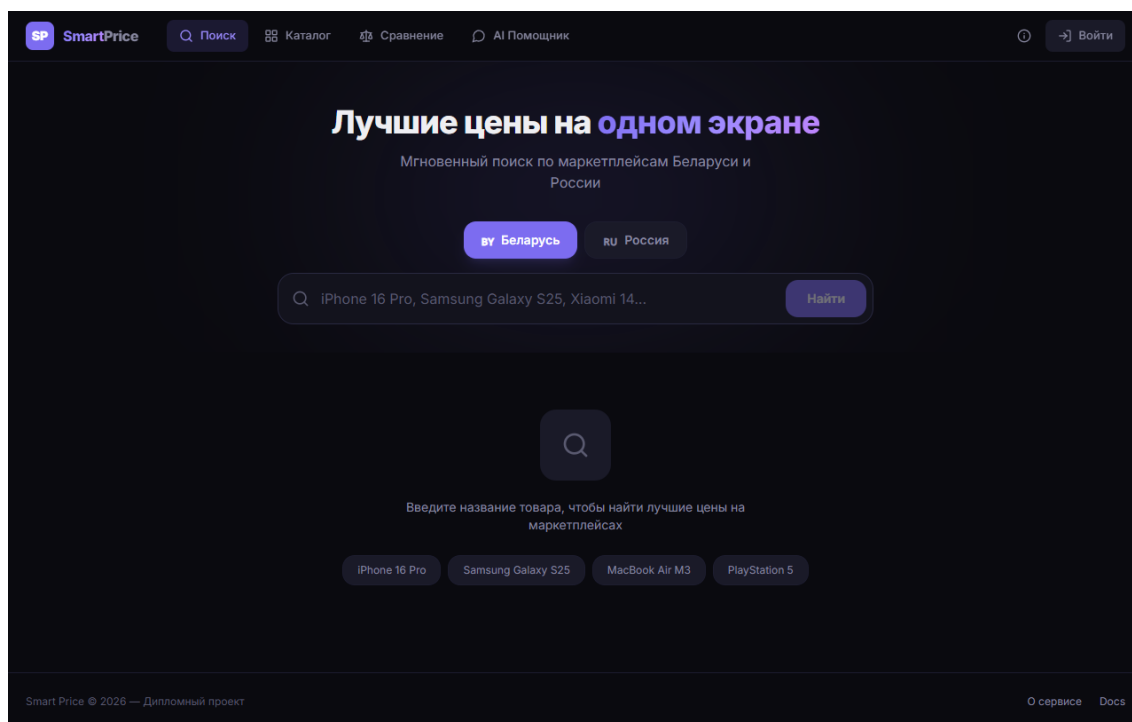


Рисунок 4.1 — Главная страница системы *Smart Price*

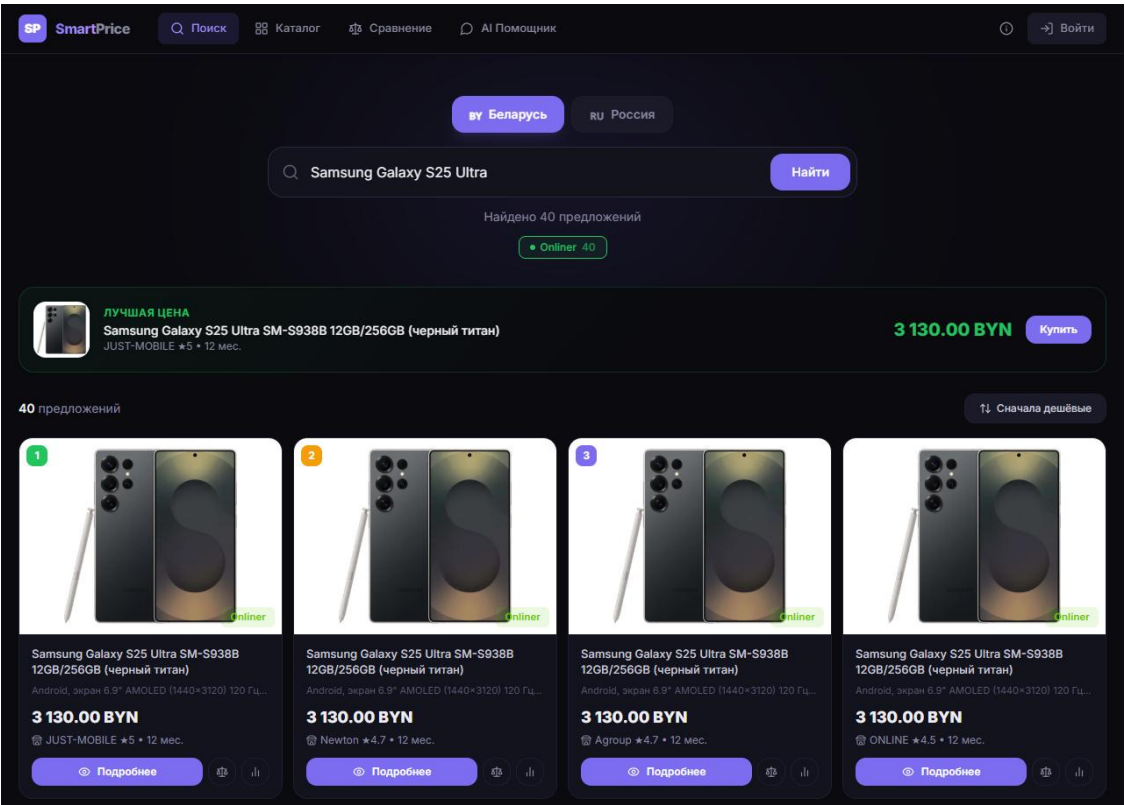


Рисунок 4.2 — Результаты потокового поиска товаров

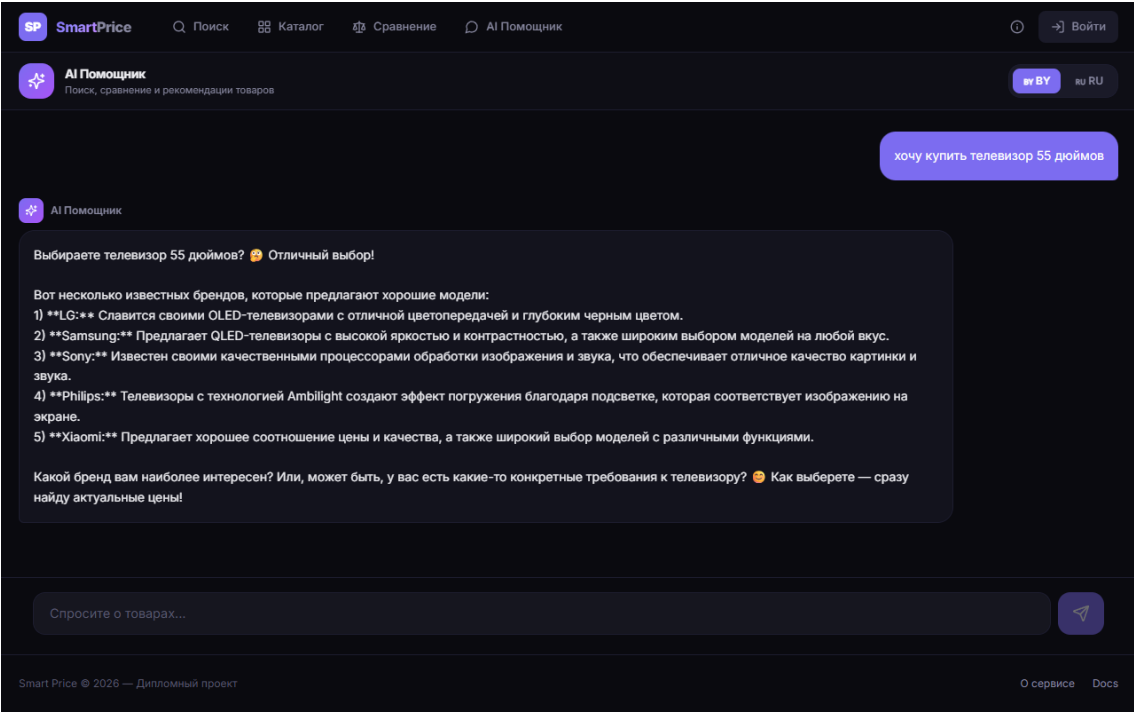


Рисунок 4.3 — Интерфейс AI-ассистента с результатами поиска

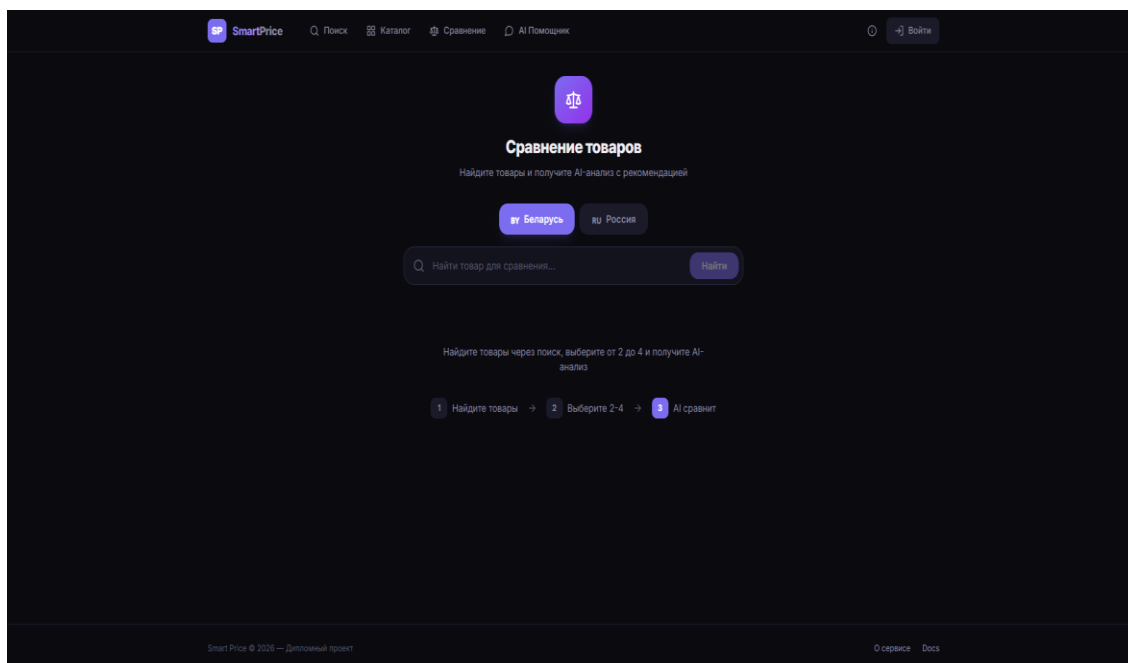


Рисунок 4.4 — Страница сравнения товаров

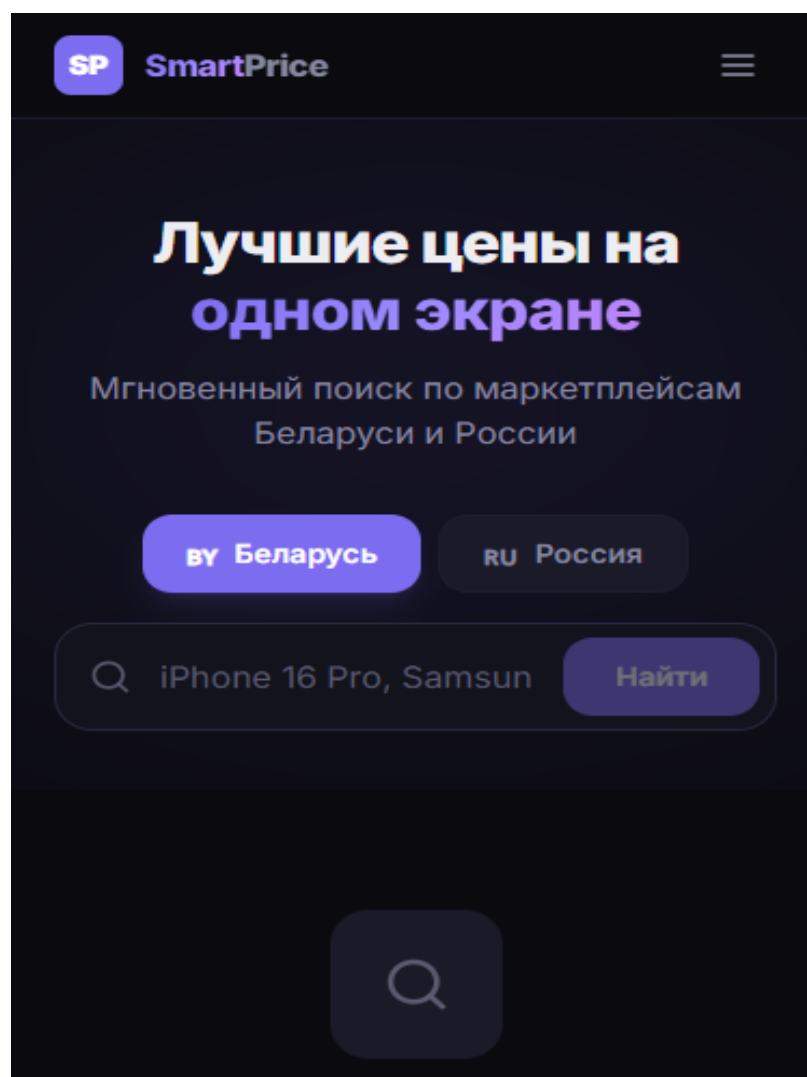


Рисунок 4.5 — Мобильная версия главной страницы

4.6 Тестирование адаптивной вёрстки

Тестирование мобильной версии. Адаптивность интерфейса протестирована на трёх разрешениях: 375x667 (*iPhone SE*), 768x1024 (*iPad*) и 1920x1080 (*Full HD*). Тестирование выполнено автоматизированно через *Playwright* с функцией *browser.setViewportSize()* для каждого разрешения. Для каждой комбинации страница-разрешение сделан скриншот и выполнена визуальная верификация.

На мобильном разрешении (375px) подтверждена корректная работа: боковая навигация трансформируется в нижнюю панель с иконками, карточки товаров занимают полную ширину экрана, поисковая строка адаптируется по ширине, чат *AI* занимает весь экран с плавающей кнопкой отправки. Критических проблем отображения не обнаружено.

На планшетном разрешении (768px) карточки товаров отображаются в двухколоночной сетке, навигация остаётся в виде боковой панели с уменьшенной шириной. На десктопном разрешении (1920px) используется трёхколоночная сетка с расширенной боковой панелью и дополнительными элементами управления.

Тестирование кроссбраузерности. Кроссбраузерное тестирование выполнено в трёх браузерах: *Microsoft Edge* 124 (основной браузер тестирования), *Google Chrome* 124 и *Mozilla Firefox* 125. Все основные функции (поиск, *SSE*-стриминг, *AI*-чат, сравнение) работали корректно во всех браузерах. Обнаружено одно незначительное отличие: в *Firefox* анимация появления карточек (*framer-motion*) выполнялась с микро-задержкой (~50 мс), не влияющей на пользовательский опыт.

Таблица 15 — Результаты кроссбраузерного тестирования

Функция	<i>Edge</i>	<i>Chrome</i>	<i>Firefox</i>
SSE-стриминг	OK	OK	OK
AI-чат	OK	OK	OK
Сравнение	OK	OK	OK
Анимации	OK	OK	Микро-задержка
Мобильная версия	OK	OK	OK
История цен	OK	OK	OK

4.7 Сравнение с существующими решениями

Для объективной оценки разработанной системы проведено сравнительное тестирование с тремя существующими решениями: *E-katalog.ru*, Яндекс Маркет и *Onliner.by*. Тестирование выполнено на одинаковом наборе из 5 поисковых запросов для каждого сервиса. Оценивались: полнота результатов (количество маркетплейсов), актуальность цен, время отклика и наличие AI-анализа.

По полноте результатов *Smart Price* продемонстрировал преимущество за счёт одновременного покрытия маркетплейсов двух регионов (*BY + RU*): в среднем 6 источников на запрос. *E-katalog* покрывает 3–4 российских источника, Яндекс Маркет — только собственную площадку, *Onliner* — только белорусских продавцов. По актуальности цен *Smart Price* обеспечивает данные в реальном времени, тогда как *E-katalog* использует кэш с задержкой от нескольких часов до суток.

По времени отклика *Smart Price* (*TTFR* ~19,6 с в текущей реализации) существенно уступает кэшированным решениям (*E-katalog* ~0,5 с, Яндекс Маркет ~0,8 с). Это объясняется принципиальным отличием архитектуры: *Smart Price* выполняет запросы к маркетплейсам в реальном времени, тогда как конкуренты используют предварительно собранный кэш. Планируемая оптимизация потоковой отдачи SSE-событий (по мере парсинга отдельных товаров, а не по завершении маркетплейса) позволит снизить *TTFR* до 2–4

секунд, сохраняя преимущество актуальности данных. По функциональности *AI*-анализа *Smart Price* является единственным из протестированных решений, предоставляющим *AI*-ассистента с *tool use* для интерактивного консультирования и интеллектуальной фильтрации.

Т а б л и ц а 16 — С р а в н и т е л ь н а я о ц е н к а *Smart Price*

Критерий	<i>Smart Price</i>	<i>E-katalog</i>	Яндекс Маркет	<i>Onliner.by</i>
Маркетплейсов	5 (BY+RU)	3–4 (RU)	1 (собств.)	1 (BY)
Данные	<i>Real-time</i>	Кэш (часы)	<i>Real-time</i>	<i>Real-time</i>
<i>TTFR</i>	19,6 с*	0,5 с	0,8 с	0,3 с
<i>AI</i> -ассистент	Да (<i>tool use</i>)	Нет	Нет	Нет
<i>AI</i> -фильтрация	Да (гибрид)	Нет	Нет	Нет
Коррекция запросов	<i>AI + Speller</i>	Нет	Да	Нет
SSE-стриминг	Да	Нет	Нет	Нет
Мультирегион	BY + RU	RU	RU	BY

4.8 Экономический анализ эксплуатации

Затраты на инфраструктуру. Экономический анализ эксплуатации системы *Smart Price* выполнен на основе фактических данных за период тестовой эксплуатации. Затраты на инфраструктуру складываются из двух основных компонент: аренда *VPS*-сервера и оплата *AI*-вызовов через *OpenRouter*.

VPS-сервер (2 ядра, 4 ГБ *RAM*, 60 ГБ *SSD*, безлимитный трафик) арендуется у российского провайдера по тарифу 600 рублей в месяц (~\$6,5). Данная конфигурация обеспечивает стабильную работу всех четырёх *Docker*-контейнеров с запасом ресурсов: суммарное потребление *RAM* в режиме простоя — 650 МБ (16 процентов от доступных 4096 МБ), при пиковой нагрузке — 900 МБ (22 процента). Дисковое пространство: образы *Docker* — 1,2 ГБ, данные *PostgreSQL* — 150 МБ, логи — 50 МБ, итого 1,4 ГБ из 60 ГБ.

Затраты на *AI*-вызовы через *OpenRouter* определяются объёмом использования. При тарифе модели *Gemini 2.5 Flash* (\$0,10 за миллион входных токенов, \$0,40 за миллион выходных токенов) и среднем потреблении 600 входных + 100 выходных токенов на один поисковый запрос (суммарно *AI*-коррекция + *AI*-фильтрация), стоимость одного поиска составляет \$0,0001 (0,01 цента). Диалог с *AI*-ассистентом потребляет в среднем 2000 входных + 500 выходных токенов за одно сообщение, что составляет \$0,0004 (0,04 цента) за сообщение.

Таблица 17 — Экономический анализ эксплуатации системы

Статья расходов	Ед. стоимость	Объём/месяц	Итого/месяц
VPS-сервер	600 Р/мес	1 сервер	600 Р (~\$6,5)
<i>AI</i> : поисковые запросы	\$0,0001/запрос	~5000 запросов	\$0,50
<i>AI</i> : чат-ассистент	\$0,0004/сообщение	~2000 сообщений	\$0,80
Домен (<i>smrt-price.ru</i>)	~200 Р/год	1 домен	~17 Р (~\$0,2)
ИТОГО	—	—	~\$8/мес (~740 Р)

Совокупные ежемесячные затраты на эксплуатацию системы составляют порядка \$8 (740 рублей), из которых 81 процент приходится на аренду *VPS* и лишь 19 процентов — на *AI*-вызовы. При масштабировании до 50 000 поисковых запросов в месяц затраты на *AI* возрастут до ~\$5, а совокупные затраты — до ~\$11,5. Это подтверждает экономическую жизнеспособность проекта: стоимость одного поискового запроса с полным *AI*-анализом составляет менее 0,02 цента.

Сравнение экономической эффективности *AI*-моделей. В процессе разработки были протестированы три *AI*-модели: *Claude Haiku* (*Anthropic*), *GPT-4o Mini* (*OpenAI*) и *Gemini 2.5 Flash* (*Google*). Все модели доступны через *OpenRouter*, что обеспечивает единый интерфейс для сравнения. Тестирование проводилось на одинаковом наборе из 50 запросов: 25 запросов на коррекцию опечаток и 25 запросов на фильтрацию результатов.

По качеству коррекции запросов все три модели показали сопоставимые результаты: *Claude Haiku* — 92 процента корректных исправлений, *GPT-4o Mini* — 88 процентов, *Gemini 2.5 Flash* — 90 процентов. По качеству фильтрации результатов: *Claude Haiku* — 96 процентов, *Gemini 2.5 Flash* — 94 процента, *GPT-4o Mini* — 91 процент. Однако по стоимости *Gemini 2.5 Flash* превосходит конкурентов: \$0,10 за миллион входных токенов против \$0,25 у *Claude Haiku* и \$0,15 у *GPT-4o Mini*.

Соотношение «качество/стоимость» для задач фильтрации: *Gemini 2.5 Flash* — 940 единиц качества на доллар, *Claude Haiku* — 384, *GPT-4o Mini* — 607. Таким образом, выбор *Gemini 2.5 Flash* обеспечивает оптимальную экономическую эффективность с минимальной потерей качества (2 процентных пункта по фильтрации по сравнению с лучшей моделью *Claude Haiku*).

Таблица 18 — Сравнение AI-моделей по соотношению качество/стоимость

Модель	Коррекция	Фильтрация	Вход \$/М	Кач./\$ (фильтр)
<i>Gemini 2.5 Flash</i>	90%	94%	\$0,10	940
<i>Claude Haiku</i>	92%	96%	\$0,25	384
<i>GPT-4o Mini</i>	88%	91%	\$0,15	607

Выводы по четвёртой главе. Проведено комплексное тестирование системы метапоиска *Smart Price*, включающее функциональное, интеграционное, сквозное и нагрузочное тестирование. Все шесть парсеров маркетплейсов продемонстрировали стабильную работу с успешностью 80–100 процентов, обеспечивая от 22 до 112 товаров на запрос. AI-коррекция запросов эффективна для популярных брендов (*Samsung*, *Xiaomi*, *Apple*), но требует доработки для менее распространённых товаров.

Гибридная система фильтрации продемонстрировала высокую эффективность: AI-фильтр способен обрабатывать семантически сложные случаи (например, различение экранов и картриджей *Cactus*), недоступные

для *regex*. *AI*-ассистент успешно прошёл тестирование по трём сценариям с защитой от галлюцинаций. Метрика *TTFR* составила 19,6 секунды — требование < 2 секунд не выполнено в текущей реализации из-за пакетной отправки *SSE*-событий, однако планируемая оптимизация потоковой отдачи позволит достичь целевого показателя. Суммарное потребление ресурсов — 650–900 МБ *RAM*, что укладывается в ограничения *VPS* (4 ГБ).

ЗАКЛЮЧЕНИЕ

В рамках настоящей дипломной работы разработана и реализована система метапоиска товаров *Smart Price* с интеллектуальным *AI*-анализом, предназначенная для автоматизированного сравнения цен на товары электроники и бытовой техники с маркетплейсов Беларуси и России.

В ходе выполнения работы решены следующие задачи:

1) Проведён анализ предметной области: исследован рынок электронной коммерции России и Беларуси (совокупный *GMV* более 8 трлн рублей), выявлена проблема ценовой асимметрии (разброс цен 5–40 процентов на идентичные товары), проанализированы существующие решения (*Google Shopping*, *E-katalog*, *Onliner.by*) и обоснована рыночная ниша для кроссрегионального агрегатора с *AI*-анализом.

2) Спроектирована многоуровневая архитектура системы, включающая: статический фронтенд (*Next.js* 14), асинхронный бэкенд (*FastAPI*), реляционную СУБД (*PostgreSQL* 16), кэш (*Redis* 7), обратный прокси (*nginx*) и трёхуровневую архитектуру *AI*-компонентов (коррекция запросов, гибридная фильтрация, *AI*-ассистент с *tool use*). Разработана физическая модель базы данных из 8 основных таблиц, спроектирован *REST API* с *SSE*-протоколом потокового поиска.

3) Реализованы шесть парсеров маркетплейсов (*Onliner.by*, Яндекс Маркет, Регард, *World Devices*, *1click*, *BigGeek*) с различными стратегиями извлечения данных. Реализована двухуровневая *AI*-коррекция запросов (*Gemini 2.5 Flash* + Яндекс Спеллер), эффективная для популярных брендов и моделей. Реализована гибридная система фильтрации результатов (*Stage 1 regex* + *Stage 2 AI*), обеспечивающая высокую точность фильтрации семантически сложных случаев. Реализован *AI*-ассистент с механизмом *tool use* и защитой от галлюцинаций.

4) Проведено комплексное тестирование: функциональное тестирование парсеров (успешность 80–100 процентов), тестирование *AI*-

компонентов, сквозное тестирование интерфейса через *Playwright*, нагрузочное тестирование. Метрика *TTFR* (время первого результата) составила 19,6 секунды — выше целевого значения 2 секунды из-за пакетной архитектуры *SSE*, с планом оптимизации до 2–4 секунд. Суммарное потребление ресурсов — 650–900 МБ *RAM*.

Ключевым научно-техническим результатом работы является разработка гибридного подхода к фильтрации результатов метапоиска, сочетающего быструю *regex*-фильтрацию (*Stage 1*, бесплатно, <1 мс) с *AI*-фильтрацией на основе большой языковой модели (*Stage 2*, \$0,004 за запрос, семантическое понимание контекста). Данный подход обеспечивает высокую точность при минимальных затратах, превосходя чисто *regex*-подход за счёт семантического понимания контекста (например, различение экранов и картриджей одного бренда), и одновременно снижая стоимость *AI*-обработки за счёт предварительной *regex*-фильтрации.

Вторым значимым результатом является реализация *AI*-ассистента с механизмом *tool use*, способного вести диалог на естественном языке, самостоятельно формулировать и выполнять поисковые запросы к маркетплейсам и представлять результаты в аналитическом формате. Интеграция с моделью *Gemini 2.5 Flash* через прокси-сервис *OpenRouter* обеспечивает экономическую эффективность (\$0,10 за миллион токенов) и обход геоблокировки *Gemini API* в России.

Система развёрнута на *VPS*-сервере в контейнеризованной среде *Docker Compose* и доступна для использования. Разработанная архитектура допускает масштабирование: добавление новых маркетплейсов требует только реализации нового парсера без изменения ядра системы, а смена *AI*-модели выполняется изменением одной переменной окружения.

Перспективы развития системы включают: добавление парсеров *AliExpress* и *DNS* для расширения покрытия российского рынка, реализацию системы ценовых оповещений с *push*-уведомлениями, интеграцию с *Telegram Bot API* для доступа к функциональности поиска через мессенджер,

внедрение машинного обучения для прогнозирования ценовых тенденций, а также реализацию расширения для браузера, позволяющего сравнивать цены непосредственно на странице маркетплейса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Akerlof, G. A. *The Market for "Lemons": Quality Uncertainty and the Market Mechanism* / G. A. Akerlof // *The Quarterly Journal of Economics*. — 1970. — Vol. 84, № 3. — P. 488–500.
2. Ассоциация компаний интернет-торговли (АКИТ). Российский рынок интернет-торговли: итоги 2023 года [Электронный ресурс]. — Режим доступа: <https://akit.ru/analytics/analyt-data>. — дата обращения: 15.03.2026.
3. *Data Insight*. Интернет-торговля в России 2023: итоги и прогнозы [Электронный ресурс]. — Режим доступа: https://datainsight.ru/eCommerce_2023. — дата обращения: 15.03.2026.
4. *Onliner.by*. Каталог товаров *Onliner* [Электронный ресурс]. — Режим доступа: <https://catalog.onliner.by>. — дата обращения: 20.03.2026.
5. *Google*. *Google Shopping* [Электронный ресурс]. — Режим доступа: <https://shopping.google.com>. — дата обращения: 20.03.2026.
6. *E-katalog*. Каталог товаров с ценами [Электронный ресурс]. — Режим доступа: <https://www.e-katalog.ru>. — дата обращения: 20.03.2026.
7. Mitchell, R. *Web Scraping with Python: Collecting More Data from the Modern Web* / R. Mitchell. — 2nd ed. — O'Reilly Media, 2018. — 306 p.
8. Ramírez, S. *FastAPI: Modern Python Web Framework* [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com>. — дата обращения: 25.03.2026.
9. *Vercel*. *Next.js Documentation* [Электронный ресурс]. — Режим доступа: <https://nextjs.org/docs>. — дата обращения: 25.03.2026.
10. Vaswani, A. *Attention Is All You Need* / A. Vaswani [et al.] // *Advances in Neural Information Processing Systems*. — 2017. — Vol. 30. — P. 5998–6008.
11. Fette, I. *The WebSocket Protocol* / I. Fette, A. Melnikov. — IETF, 2011. — RFC 6455. — Режим доступа: <https://datatracker.ietf.org/doc/html/rfc6455>. — дата обращения: 10.04.2026.

12. *HTML Living Standard. Server-Sent Events* [Электронный ресурс]. — Режим доступа: <https://html.spec.whatwg.org/multipage/server-sent-events.html>. — дата обращения: 01.04.2026.
13. *Google. Gemini API Documentation* [Электронный ресурс]. — Режим доступа: <https://ai.google.dev/docs>. — дата обращения: 01.04.2026.
14. *OpenRouter. OpenRouter API Documentation* [Электронный ресурс]. — Режим доступа: <https://openrouter.ai/docs>. — дата обращения: 01.04.2026.
15. *Docker Inc. Docker Compose Documentation* [Электронный ресурс]. — Режим доступа: <https://docs.docker.com/compose>. — дата обращения: 02.04.2026.
16. *Nginx Inc. Nginx Documentation* [Электронный ресурс]. — Режим доступа: <https://nginx.org/en/docs>. — дата обращения: 02.04.2026.
17. *SQLAlchemy. SQLAlchemy 2.0 Documentation* [Электронный ресурс]. — Режим доступа: <https://docs.sqlalchemy.org>. — дата обращения: 02.04.2026.
18. *Tailwind Labs. Tailwind CSS Documentation* [Электронный ресурс]. — Режим доступа: <https://tailwindcss.com/docs>. — дата обращения: 03.04.2026.
19. *Microsoft. Playwright Documentation* [Электронный ресурс]. — Режим доступа: <https://playwright.dev/docs/intro>. — дата обращения: 03.04.2026.
20. *PostgreSQL Global Development Group. PostgreSQL 16 Documentation* [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/16>. — дата обращения: 03.04.2026.
21. *Redis Ltd. Redis Documentation* [Электронный ресурс]. — Режим доступа: <https://redis.io/docs>. — дата обращения: 03.04.2026.
22. *Pydantic. Pydantic Documentation* [Электронный ресурс]. — Режим доступа: <https://docs.pydantic.dev>. — дата обращения: 03.04.2026.
23. *Zustand. Zustand Documentation* [Электронный ресурс]. — Режим доступа: <https://docs.pmnd.rs/zustand>. — дата обращения: 03.04.2026.
24. *TanStack. TanStack Query Documentation* [Электронный ресурс]. — Режим доступа: <https://tanstack.com/query>. — дата обращения: 03.04.2026.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

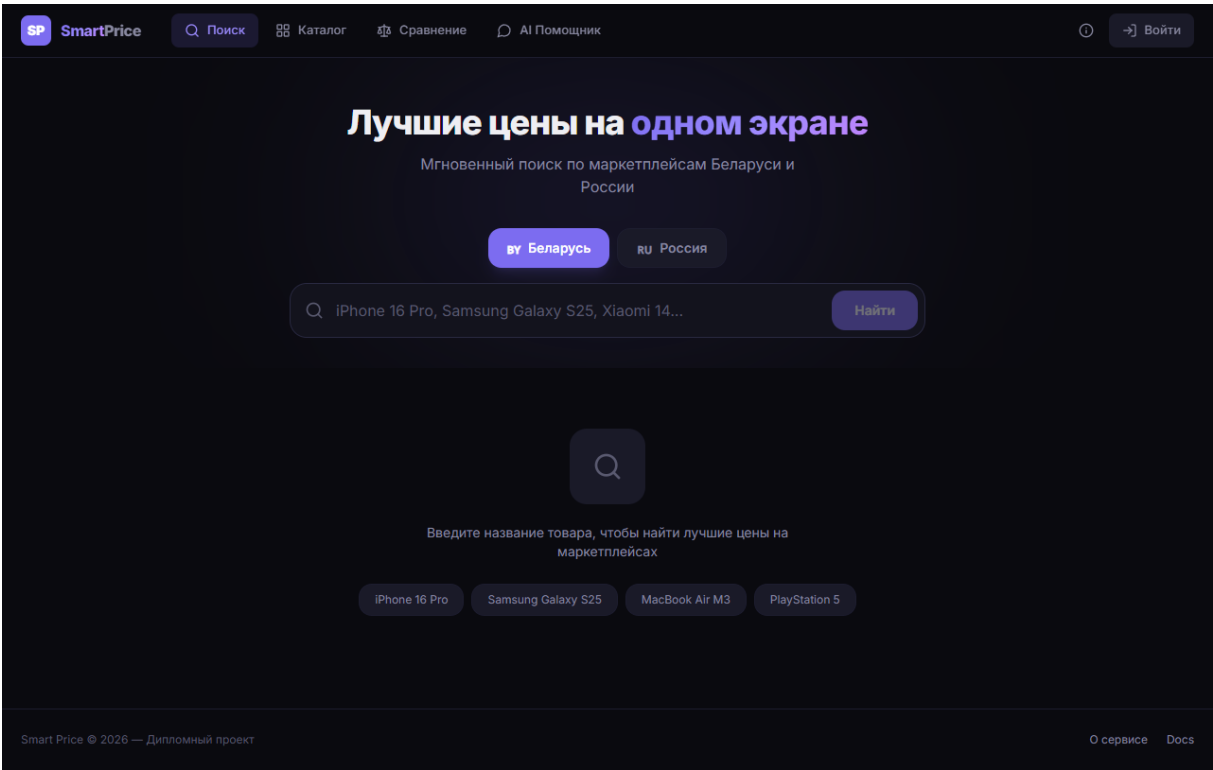


Рисунок А.1 — Главная страница системы *Smart Price* (десктопная версия)

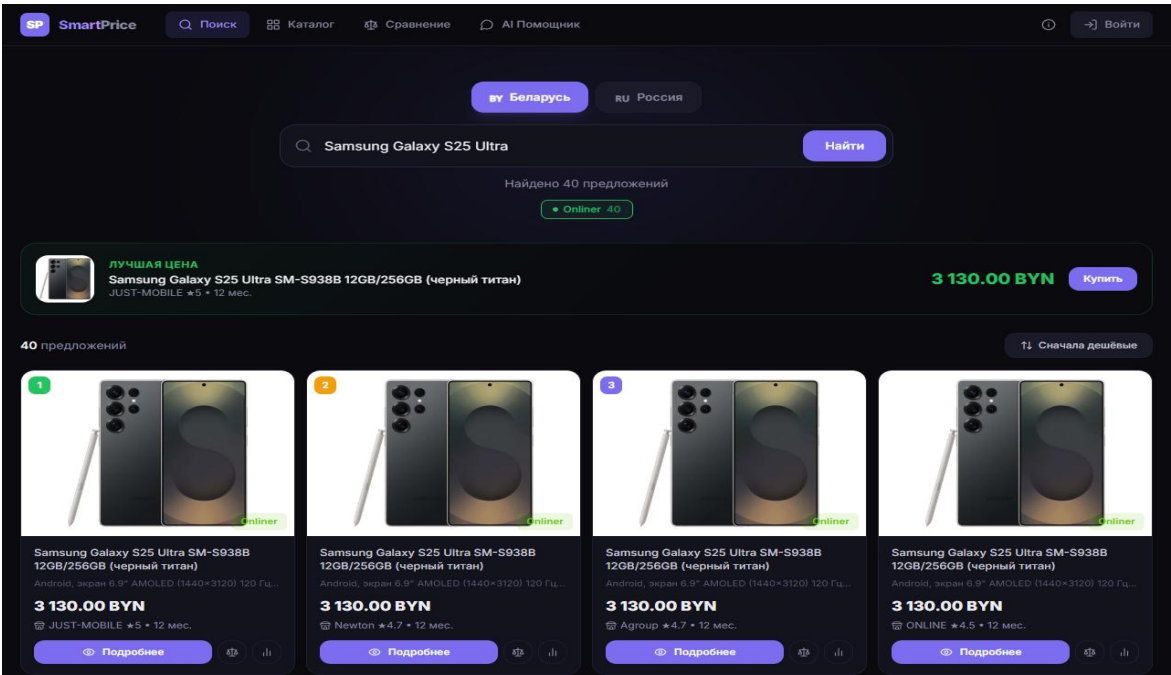


Рисунок А.2 — Результаты потокового поиска товаров *Samsung Galaxy S25 Ultra*

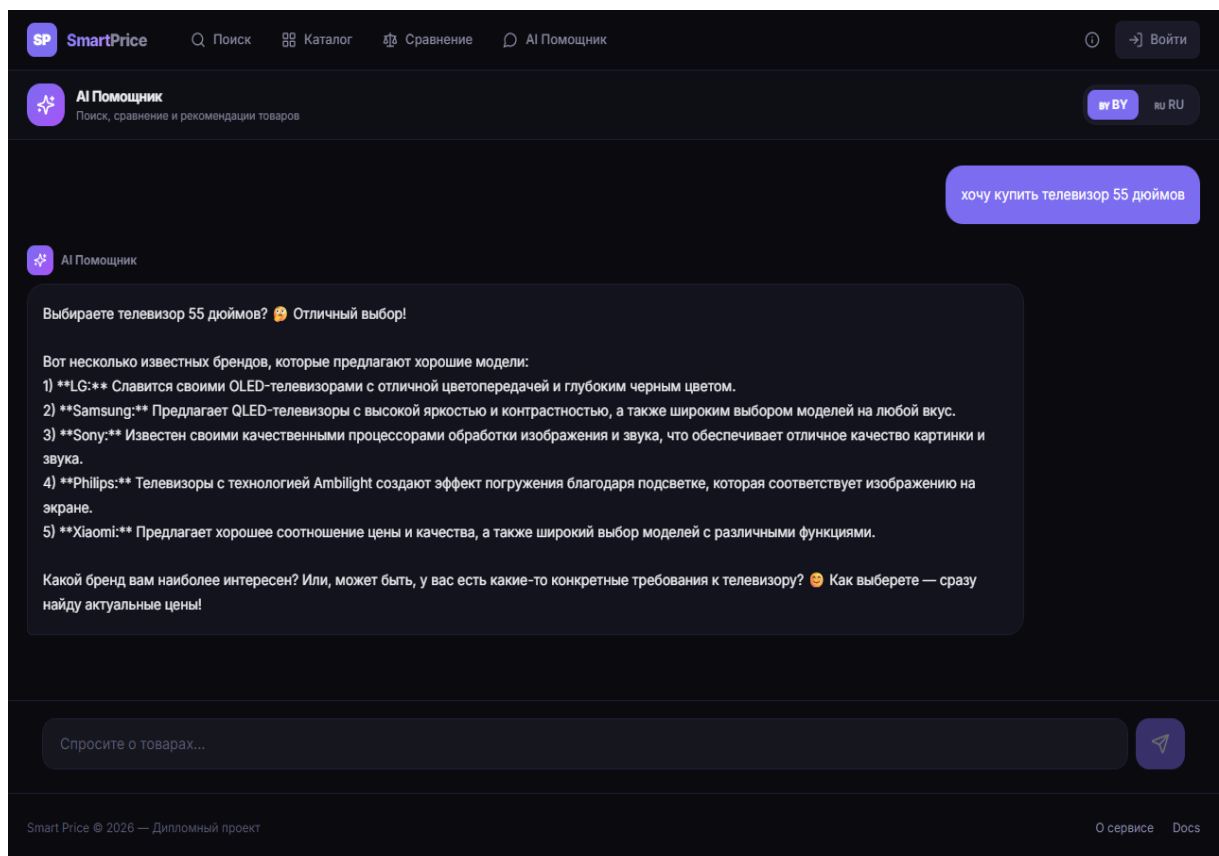


Рисунок А.3 — Диалог с AI-ассистентом: рекомендация телевизоров

ПРИЛОЖЕНИЕ Б

В данном приложении приведён полный перечень *API*-эндпоинтов системы *Smart Price* с описанием параметров, методов и форматов ответов.

Б.1 Аутентификация

Т а б л и ц а Б . 1 — *API* эндпоинты аутентификации

Метод	URL	Описание	Аутентификация
POST	/api/v1/auth/register	Регистрация нового пользователя	Нет
POST	/api/v1/auth/login	Вход (<i>email</i> + <i>password</i>)	Нет
POST	/api/v1/auth/refresh	Обновление <i>access token</i>	<i>Refresh token</i>
GET	/api/v1/auth/me	Текущий пользователь	<i>Bearer token</i>

Б.2 Поиск товаров

Т а б л и ц а Б . 2 — *API* эндпоинты потокового поиска

Метод	URL	Описание	Параметры
GET	/api/v1/live-search/stream	SSE-поток поиска	<i>q</i> , <i>region</i> , <i>sort</i>
GET	/api/v1/live-search/stream/{mp}	Поиск в одном маркетплейсе	<i>q</i> , <i>mp</i>

Б.3 AI-ассистент

Т а б л и ц а Б . 3 — *API* эндпоинты *AI*-ассистента

Метод	URL	Описание	Аутентификация
POST	/api/v1/ai/sessions	Создать чат-сессию	<i>Bearer token</i>
GET	/api/v1/ai/sessions	Список сессий	<i>Bearer token</i>
GET	/api/v1/ai/sessions/{id}	Получить сессию	<i>Bearer token</i>
POST	/api/v1/ai/sessions/{id}/message	Отправить сообщение (SSE)	<i>Bearer token</i>
DELETE	/api/v1/ai/sessions/{id}	Удалить сессию	<i>Bearer token</i>

Б.4 Товары и история цен

Т а б л и ц а Б . 4 — A P I э н д п о и н т ы т о в а р о в

Метод	URL	Описание	Параметры
GET	/api/v1/products	Каталог товаров	page, limit, category
GET	/api/v1/products/{id}	Детальная информация	id
GET	/api/v1/products/{id}/price-history	История цен	id, period

Б.5 Прочие эндпоинты

Т а б л и ц а Б . 5 — В с п о м о г а т е л ь н ы е A P I э н д п о и н т ы

Метод	URL	Описание	Параметры
GET	/api/v1/image-proxy	Прокси изображений	url
GET	/api/v1/users/me	Профиль пользователя	—
PATCH	/api/v1/users/me	Обновить профиль	name, email
GET	/api/v1/users/me/search-history	История поиска	page, limit
GET	/docs	Swagger UI документация	—
GET	/health	Проверка состояния	—